



**GÁBOR
DÉNES
EGYETEM**

Mérnökinformatika

SZAKDOLGOZAT

**Open-Source Implementation of „Real-Time Lip Sync for Live 2D
Animation”**

Belle Dániel

245/2024

Table of Contents

1 Background	5
2 An Overview	6
3 Planning.....	8
4 The Data	11
4.1 Audio Feature Vector	11
4.2 Rhubarb lip-sync.....	12
4.3 SPHINX-II.....	15
4.4 The Mel-frequency Cepstrum Coefficient.....	16
4.4.1 Pre-emphasis	17
4.4.2 Signal Framing and Windowing.....	18
4.4.3 Discrete Fourier Transform	18
4.4.4 Mel Filter Bank	18
4.4.5 Discrete Cosine Transform.....	19
4.5 Energy.....	20
4.6 Counteracting large audio changes.....	20
4.7 Librosa	21
5 Feature Extraction	23
5.1 Combining the data.....	26
6 Data Loading.....	28
6.1 Data Loading During Training	28
6.2 Data Loading During Inference	31
7 The model.....	32
7.1 Recurrent Neural Networks	32
7.2 Long Short Term Memory	33

7.3 Loss Function	36
7.4 Cross Entropy Loss.....	36
7.5 The PyTorch framework.....	37
7.6 Model structure.....	38
7.7 Training.....	40
7.7.1 Baseline	40
7.7.2 Experiments.....	41
7.7.3 Ground Truth Testing	56
8 Animation	57
8.1 Interface	58
8.2 Noise reduction.....	58
8.3 Silence mode.....	59
8.4 Reducing prediction noise	59
8.5 Downsampling to 24 FPS	60
8.6 Single frame viseme removal	60
9 Limitations	61
10 Further development	62
11 System specifications and tools.....	64
12 Conclusion.....	65
13 References	66
14 Table of Figures.....	69
15 Table of Tables	70
Table of Appendices	71
Appendices	72

Abstract

The goal of this thesis is to create an open-source implementation of a real-time lip sync for live 2D animation method using a deep learning model that utilizes LSTM (Long Short Term Memory), as researched by Deepali Aneja and Wilmot Li in their paper (Aneja & Li, 2019). The pair of researchers shared an overview of their methods and experiment results, however the code and the data they used is not accessible for free. Therefore, I decided to recreate the software based on the paper, with my own training dataset. I chose to tackle this problem because it was brought up to me by my Artificial Intelligence teacher, Dávid Jakab, as a small challenge, however, upon a closer look, I realized that this task is a bit more complicated than it seemed at first. The main complication is the lack of budget to have human animators help create the dataset, which is an expensive and time-consuming process. Having realized this, I looked for alternatives, eventually coming to the conclusion that the output of a free non real time lip-sync software could replace the animators. During the creation process I examined differences between my method, using the software generated dataset, and the researchers' method who used human animators. In this examination, I identified several factors that contributed to the resulting animation's shortcomings, and I propose solutions for these, as well as areas that can be further explored to achieve a more realistic result.

1 Background

Live 2D animation has been on the rise in entertainment as a means for one or more audience members to interact with the character on screen, which is voiced by an actor. This form of entertainment rose to prevalence in recent decades, from examples like Homer Simpson engaging with viewers during live segments of *The Simpsons*. In a notable 2016 episode, Homer answered fan questions live, utilizing motion capture technology to animate his responses in real-time (Aneja & Li, 2019)

More recently, the rise of VTubers, virtual YouTubers, has exemplified this interactive animation form. VTubers employ animated avatars to represent themselves during live streams, allowing for dynamic engagement with their audience. This phenomenon has seen substantial growth, with streaming platforms like Twitch reporting a 350% increase in VTuber content viewership in 2022. (Norman, 2024)

In the realm of live entertainment, concerts and theater productions have increasingly incorporated live 2D animation to create interactive and immersive performances. Animated characters are often projected onto screens alongside live performers, enabling real-time interactions that enhance the audience's experience. This integration of live 2D animation in various entertainment venues demonstrates its versatility and growing popularity, allowing for innovative engagement that blurs the line between digital and live performance.

2 An Overview

The goal is to create a system that takes a live audio input and outputs a sequence of visemes, encoded as letters, which are then mapped to a set of images and then displayed to create an animation of a 2D character speaking. A viseme (or mouth cue) is a part of a set of images that animators use to make a 2D character talk. Each viseme represents a state of the lips when making a certain sound. It is common to use 12 visemes in a set, however this is not exclusive. For example, Rhubarb lipsync, the software used in this thesis, uses 9 visemes in its extended set (Wolf, 2022).

To achieve this, I will generate a dataset, which will be used for training an LSTM model which takes audio feature vectors as inputs and outputs visemes. The raw data source will be the TIMIT Acoustic-Phonetic Continuous Speech Corpus (Garofolo, 1993). This corpus of data contains audio files, from which I extract audio features to build the input vector. For training, labels are also needed, which are generated by putting the audio files through Rhubarb lip sync (Wolf, 2022). The software generates visemes for the audio, and a timestamp for each transition. Based on the timestamps, I concatenate the labels to each individual set of audio features. The features combined with labels are then stored in JSON files, separated by which recording they belong to.

The LSTM model is then trained on this data, then saved. This saved model is then loaded when running the lip-sync app and is set up to provide continuous outputs based on live microphone audio that is converted to input vectors in real time. These outputs are then mapped to a set of image file names, which are displayed according to the current prediction.

Most of the methods are described in the original paper, however some details were up for interpretation, and they didn't include the tools used (Aneja & Li, 2019). Except for a reference to the Torch framework, which means their solution was implemented in Matlab (Collobert, Kavukcuoglu, & Farabet, 2011). I decided to use the Python programming language along with built-in and external packages. Python is popular in modern machine learning applications due to its readability and accessible nature. Python enables quick prototyping and development and is easy to modify and iterate on since it's an interpreter language, which does not require building before running.

This project is possible in many other programming languages with machine learning libraries, technically there is no other reason to use it in this case over other languages, however it has become industry standard, and I personally have experience with it.

An aspect of this project that has multiple approaches is the dataset generation. Better results could possibly be achieved by using paid lip-sync software to label the recordings, or an entirely different set of audio files could be used. Specifically, when creating a model for different languages. Due to the recognizer used in Rhubarb lip sync, and the fact that the TIMIT data corpus recordings are in English, this model will perform best when taking English language inputs. This is due to the fact that the LSTM architecture relies heavily on lookahead, essentially anticipating the next inputs, so if the model is trained on one language, it won't be able to predict the next sounds or words in a language that is unknown to it. A multilingual approach is possible by using a generic recognizer and including multiple languages in the dataset, however I do not have access to recordings of enough languages.

Similar solutions to live 2D lip-sync exist, however, only Adobe Ch (Character Animator) comes close to this concept. The software's website describes that visemes are derived from audio only, but neutral, smiling, and surprised expressions come from camera input. (Adobe Inc., 2024)

It is important to note that the original paper's authors describe the capabilities of Adobe Ch as:

“Adobe Character Animator (Ch) — the predominant live 2D animation tool — uses face tracking to translate a performer’s facial expressions to a cartoon character and keyboard shortcuts to enable explicit triggering of animated actions, like hand gestures or costume changes.” (Aneja & Li, 2019)

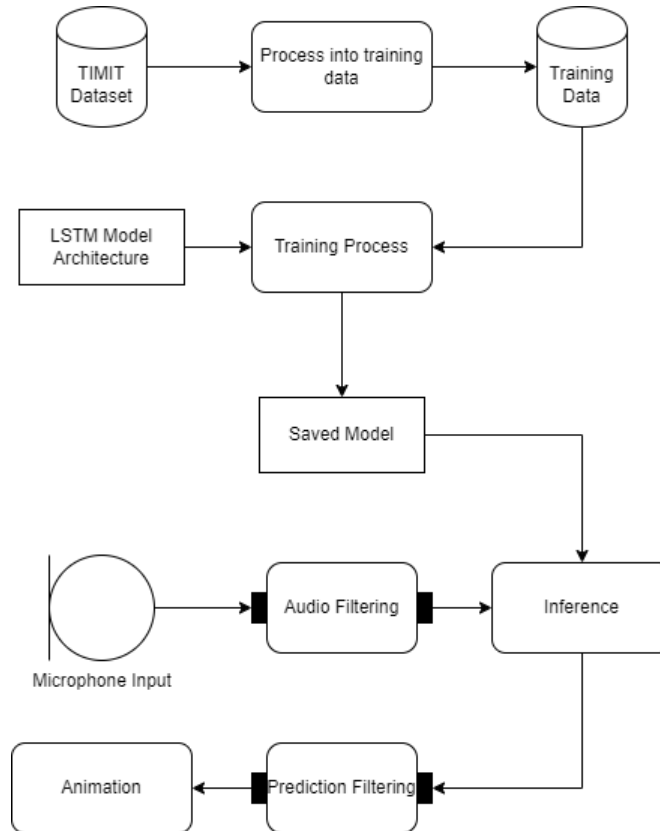
No live lip-sync functionality is mentioned as of October 2019. Wilmot Li, one of the authors of the paper, is a Principal Scientist at Adobe Research (Adobe Inc., 2024). Adobe does not reveal the inner workings of the new live lip-sync function of Adobe Ch, however it is reasonable to assume that it's based on this LSTM solution, complimented by video input.

Another similar software is Motion LIVE 2D, formerly part of CrazyTalk by Reallusion. This software uses webcam and motion capture input to generate speech animation along with motion animations as well, no mention of an audio-only option is mentioned on their website (Reallusion, 2024).

3 Planning

The planning phase of this project was simple to structure due to the existing research, however, experimenting with different methods played a significant role in the creation process.

Figure 1: High-level overview of the entire process



Source: Self made

The structure of the working process of the application is pictured in Figure 1, from data creation, through inference, to the final animation. The structure and the code are both built to be modular. The data source, for example, can be switched out to an entirely different dataset, as long as it consists of human speech recordings with little to no noise in waveform file format. This was especially useful because I experimented with a different data source as well, and when I acquired the TIMIT dataset, I was able to use it straight away.

The alternate data source I experimented with were audiobooks downloaded from free sources. The aim was to acquire audiobook recordings between 10 to 20 minutes in length and extract features from them. While this method made it easy to gather a large amount of data in terms of

usable minutes of audio, it lacked speaker variety. A dataset with a decent speaker variety is crucial to this application, since it allows the model to adapt to unknown voices when in use. Therefore, using long recordings makes it inefficient to reach the speaker variety that the TIMIT dataset has. The TIMIT dataset consists of 70% male and 30% female speakers and was collected from 8 dialect regions. These regions are described as (Garofolo, 1993):

- New England
- Northern
- North Midland
- South Midland
- Southern
- New York City
- Western
- Army Brat (moved around)

This means that using this dataset will adapt the model best to North American speakers, which is not a significant problem as Rhubarb lip sync is set use a recognizer optimized for the English language (detailed later). The upside of a varied North American dataset is that it includes different ethnicities as well, which adds to the diversity of the speakers, allowing the model to generalize even better. Overall, the variety and the standardized nature of the TIMIT corpus makes it an excellent choice for a data source.

The next step is extracting the features that will be used to train the model from the waveform files. The training data is generated in 3 steps. First, each audio file goes through feature extraction, then Rhubarb lip sync is run on every recording to generated visemes. This creates 2 files for each recording, which are then combined in the third step, creating the final form of the training dataset. Due to the fact that my data source is 10 times larger than the in the original paper, I opted not to augment the data, as the amount seems more than sufficient.

After this, the model with an LSTM architecture is trained on the data. I opted to follow this method because the data is sequential, and LSTMs are well-suited for sequential data as they can leverage temporal context. This capability allows the model to consider the temporal relationships between sounds and predict corresponding visemes, making it effective for tasks involving continuous audio

streams. Additionally, the simplicity of a single-layer, unidirectional LSTM with relatively few parameters helps achieve real-time performance while maintaining prediction accuracy, even with limited computational resources.

Post-training, the model is saved as a “. pth” file, which utilizes Python’s pickle module to save the state of the model (PyTorch Foundation, 2024). This model file is then loaded during inference.

Inference is set up as follows: first, the audio input stream is set up, then the model is loaded and set up to provide a stream of predictions based on the input at 100Hz. The output stream then goes through a series of filters and is subsampled to 24Hz. This output stream then goes through a module that renders the animation on a GUI set up by tkinter. The rendering works by associating visemes to image file names and displaying them when the viseme related to them is predicted.

I decided to use the tkinter module because it is simple to set up, it does not require any other external tools or programs and works purely in python. The animation produced in this thesis is purely for demonstration, not particularly practical. However, since the prediction stream consists of only visemes, it’s simple to implement in a variety of systems. As an experiment, I have created a web API using Flask, however I did not use it for the demonstration.

This is the overview of the application; it is made to be a simple demonstration of a modular system to be applied in larger systems. The output stream can be augmented by supplementary modules, like video input for silence detection, a system to detect laughter or singing, or it can be incorporated into 3D solutions through Blender for example.

4 The Data

In order to train the network, I needed sufficient training data. I created the input vectors from the audio features of recordings of individual sentences, combined with their respective viseme sequences. To ensure that the model will be able to adapt to different speakers, I needed a dataset that covers a wide range of dialects. The TIMIT Acoustic-Phonetic Continuous Speech Corpus was perfect for this.

“The TIMIT corpus includes time-aligned orthographic, phonetic and word transcriptions as well as a single channel, 16-bit, 16kHz speech waveform file for each utterance. The TIMIT corpus transcriptions have been hand verified. Test and training subsets, balanced for phonetic and dialectal coverage, are specified. Tabular computer-searchable information is included as well as written documentation. Speaker metadata includes gender, dialect, birth date, height, race, and education level. Of the 630 speakers, about 70% are men and 30% are women.” (Garofolo, 1993)

I am only using the .wav files included in the dataset as the model only gets an audio input. There’s one advantage I have over the original research, and that is the fact that I am using software to assign visemes to these audio files. The two researchers hired professional animators to draw visemes for a fraction of the sentences contained in the TIMIT dataset. This is because of time and cost limitations.

However, I am using a different approach as I do not have the resources to hire animators. I used open-source software called Rhubarb lip-sync (Wolf, 2022). This piece of software allows me to automate the viseme labeling process for each recording. Via this method, I am able to label the entire dataset and use it as a whole, *potentially* making up for the lack in quality since I am not using human perception for the task. However, this limits the quality of outputs to the performance of this software, which also only recognizes 9 visemes instead of 12, used in the original research.

4.1 Audio Feature Vector

The audio feature vector used as an input for the model consists of four parts:

- 13 Mel-frequency cepstrum coefficients
- 1 log energy
- 13 Mel-frequency cepstrum coefficient deltas

- 1 delta log energy

Additionally, the label, used for supervised learning, which is generated by Rhubarb lip-sync in the training dataset generation step.

Figure 2: “The audio feature a consists of MFCC, log mean energy, and their first temporal derivatives”



Source: (Aneja & Li, 2019)

Obtaining this $28 + 1$ dimensional vector is an essential part of this project, as the results will only be as good as the training data allows. This process is described below.

4.2 Rhubarb lip-sync

“Rhubarb Lip Sync allows you to quickly create 2D mouth animation from voice recordings. It analyzes your audio files, recognizes what is being said, then automatically generates lip sync information” (Wolf, 2022)

“Rhubarb Lip Sync can use between six and nine different mouth positions. The first six mouth shapes ($\textcircled{A}$ - $\textcircled{F}$) are the basic mouth shapes and the absolute minimum you have to draw for your character. These six mouth shapes were invented at the Hanna-Barbera studios for shows such as Scooby-Doo and The Flintstones. Since then, they have evolved into a de-facto standard for 2D animation, and have been widely used by studios like Disney and Warner Bros.” (Wolf, 2022)

I am using the extended viseme set, to be closer to the results of the 12 viseme method. There are also two recognizers I can choose from that the tool includes. One is called PocketSphinx, which only works on English language recordings. The other one is the phonetic recognizer, which works regardless of language but may be less accurate. Since my training data is all in English, I opted for PocketSphinx. I specified the output format to be in JSON, as it is easy to read and work with in general. It is also useful to note that this tool calls visemes “mouthcues”.

The process of obtaining viseme sentences is as follows: First, I organize all the .wav recordings into a folder, then, since rhubarb lip-sync is a command line tool, I construct a command to run the tool on each recording with my desired parameters. The output for each recording looks like this:

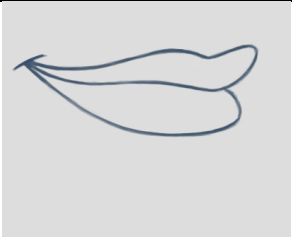
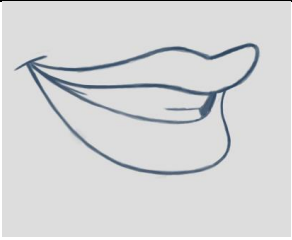
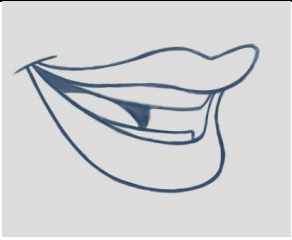
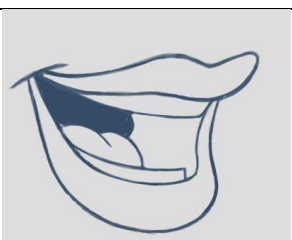
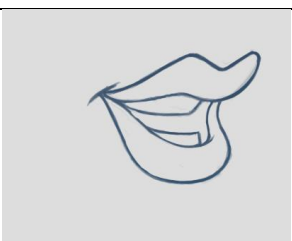
```
{
  "metadata": {
    "soundFile": "file_path",
    "duration": 3.96
  },
  "mouthCues": [
    { "start": 0.00, "end": 0.23, "value": "X" },
    { "start": 0.23, "end": 0.48, "value": "B" },
    { "start": 0.48, "end": 0.60, "value": "X" },
    { "start": 0.60, "end": 0.80, "value": "B" },
    { "start": 0.80, "end": 0.94, "value": "C" },
    { "start": 0.94, "end": 1.15, "value": "B" },
    { "start": 1.15, "end": 1.29, "value": "D" },
    { "start": 1.29, "end": 1.50, "value": "B" },
    { "start": 1.50, "end": 1.71, "value": "F" },
    { "start": 1.71, "end": 1.78, "value": "B" },
    { "start": 1.78, "end": 1.92, "value": "C" },
    { "start": 1.92, "end": 2.41, "value": "B" },
    { "start": 2.41, "end": 2.48, "value": "F" },
    { "start": 2.48, "end": 2.69, "value": "D" },
    { "start": 2.69, "end": 2.83, "value": "B" },
    { "start": 2.83, "end": 2.90, "value": "F" },
    { "start": 2.90, "end": 2.97, "value": "E" },
    { "start": 2.97, "end": 3.04, "value": "B" },
    { "start": 3.04, "end": 3.46, "value": "E" },
    { "start": 3.46, "end": 3.67, "value": "C" },
    { "start": 3.67, "end": 3.81, "value": "B" },
    { "start": 3.81, "end": 3.96, "value": "X" }
  ]
}
```

It includes metadata, which are the path to the audio file and the duration of the file, and the visemes along with their start and end timestamps.

For demonstration purposes, I am using the extended set of visemes provided in Rhubarb lip sync's documentation. To clarify, a viseme is a picture of the mouth when making certain sounds. In this

case, I use a more compact, 9 viseme set, which means that there are overlaps as the set reuses the same picture for multiple sounds, since they look similar when pronounced.

Table 1: Rhubarb viseme set

A		Closed mouth for the “P”, “B”, and “M” sounds. This is almost identical to the ⑩ shape, but there is ever-so-slight pressure between the lips.
B		Slightly open mouth with clenched teeth. This mouth shape is used for most consonants (“K”, “S”, “T”, etc.). It’s also used for some vowels such as the “EE” sound in bee .
C		Open mouth. This mouth shape is used for vowels like “EH” as in men and “AE” as in bat . It’s also used for some consonants, depending on context. This shape is also used as an in-between when animating from ⑩ or ⑪ to ⑬.
D		Wide open mouth. This mouth shape is used for vowels like “AA” as in father .
E		Slightly rounded mouth. This mouth shape is used for vowels like “AO” as in off and “ER” as in bird . This shape is also used as an in-between when animating from ⑬ or ⑭ to ⑮. Both ⑬⑮⑯ and ⑭⑮⑯ should result in smooth animation.

F		Puckered lips. This mouth shape is used for “UW” as in you , “OW” as in show , and “W” as in way .
G		Upper teeth touching the lower lip for “F” as in for and “V” as in very .
H		This shape is used for long “L” sounds, with the tongue raised behind the upper teeth. The mouth is at least as far open as in ③, but not quite as far as in ④.
X		Idle position. This mouth shape is used for pauses in speech. It is almost identical to ①, but with slightly less pressure between the lips: For ⑤, the lips should be closed but relaxed.

Source: (Wolf, 2022)

4.3 SPHINX-II

The usage of rhubarb-lipsync includes the usage of PocketSphinx (Wolf, 2022) which is based on the SPHINX-II system developed at Carnegie Mellon University (Huggins-Daines, 2024). This system is now far from state-of-the-art, however it’s open source and available for free and this application of the system produces good results.

The speech recognition system builds on the earlier SPHINX-I model to achieve speaker-independent continuous speech recognition. Unlike systems limited by speaker-specific adaptation, SPHINX-II is designed to recognize speech from a variety of users without customized training, broadening its applicability in real-world settings. Key improvements include advanced acoustic modeling techniques and a refined language model, which enable it to handle a larger vocabulary and complex sentence structures compared to SPHINX-I.

SPHINX-II employs a Hidden Markov Model (HMM) framework that uses a combination of semi-continuous and continuous density HMMs. The semi-continuous model is used to speed up processing by leveraging a shared codebook, while continuous density models improve recognition precision by capturing detailed acoustic features. A newly designed search algorithm optimizes recognition speed, allowing the system to support a larger vocabulary without a proportional increase in computational demands. This combination of model types and an efficient search algorithm enhances SPHINX-II's capability to handle diverse speech patterns and perform well across various acoustic environments.

A codebook is a collection of quantized feature vectors derived from training data, used to represent various phonetic components of speech in a compact form. Each feature vector in the codebook is essentially a prototype, representing similar acoustic characteristics across speech samples. When analyzing incoming audio, the system maps segments of speech to the closest matching vectors in the codebook, reducing the need to process each acoustic feature individually.

By referencing these shared codebook vectors (rather than unique, detailed feature vectors for every instance), the system can speed up calculations while maintaining adequate detail for accurate recognition. This is particularly helpful in semi-continuous HMMs, where each phoneme's acoustic model relies on shared codebook entries to approximate the range of sounds in speech. Thus, the codebook serves as a computationally efficient mechanism to approximate complex acoustic patterns, balancing processing load and recognition accuracy.

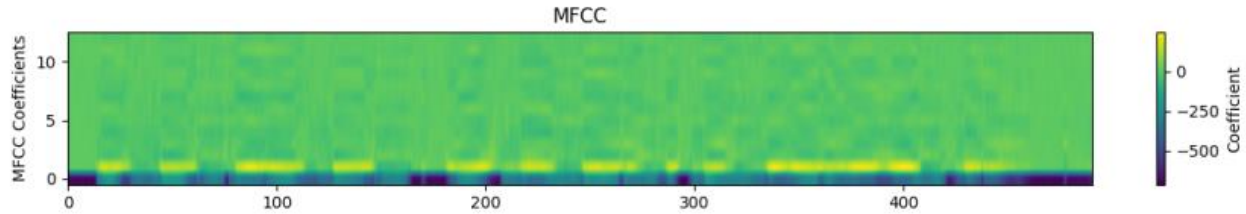
The system integrates both acoustic and language models in a unified probabilistic framework, improving word recognition accuracy and enhancing its ability to manage language variability. SPHINX-II's architecture is modular, which allows adaptation to different languages and dialects, broadening its potential applications. Importantly, the system is designed to run on commercially available hardware, making it feasible for deployment beyond experimental environments. (Huang, et al., 1993)

4.4 The Mel-frequency Cepstrum Coefficient

Mel-frequency cepstral coefficients (MFCC) are used as features in the feature vector fed to the model. Used in audio processing, the Mel-frequency Cepstrum (MFC) represents the short-term

power spectrum of a sound. These are used to represent the spectrum closer to how a human would perceive it, therefore making it useful for speech processing.

Figure 3: Example of MFCC cepstrum



Source: Code generated

Speech audio signals have visible structures in the log-spectrum, the cepstrum [Figure 3] can be obtained via an algorithm that involves applying windowing to a signal, then using time-frequency transform on it, followed by taking the logarithm of the absolute value and then applying time-frequency transform to it again. The result is called cepstrum because it is the backwards transform of „spectrum”, frequency also turns into „quefrequency” in a similar fashion (Bäckström, et al., 2022).

MFCCs are essential features used to analyze and capture the spectral characteristics of audio signals, especially for applications in speech and sound recognition. These coefficients transform audio data into a compact set of values that encode perceptually relevant information, mimicking how humans perceive frequencies. MFCCs are particularly valued in audio processing for their ability to represent sound characteristics in a way that is meaningful for machine learning and statistical modeling which is why the researchers of the original paper chose to use these features (Aneja & Li, 2019). The process of calculating MFCCs involves several sequential steps that take raw audio signals through transformations in both time and frequency domains. Understanding this process is important to potentially further improve the model by changing what features the input feature vector contains.

4.4.1 Pre-emphasis

The first step in MFCC calculation is pre-emphasis, where a high-pass filter with a setting of $[1, -0.97]$ is applied to the signal to amplify high-frequency components. This is essential because, in typical audio signals, lower frequencies tend to dominate. This filter helps balance the frequency spectrum, ensuring that high frequencies are adequately represented in the subsequent analysis (Inkeaw, 2023), (Abdul & Al-Talabani, 2022).

4.4.2 Signal Framing and Windowing

After pre-emphasis, the signal is segmented into short frames. Audio signals are non-stationary, meaning their properties change over time, so dividing them into frames (typically 20-40 milliseconds each) allows for a quasi-stationary analysis of these shorter segments. Following framing, a windowing function (often a Hamming window) is applied to each frame to reduce edge effects that can introduce spectral distortions (Abdul & Al-Talabani, 2022).

4.4.3 Discrete Fourier Transform

The Discrete Fourier Transform (DCT) is used most commonly to calculate the power spectrum, a process where each frame gets converted from the time domain to the frequency domain. The calculation for DCT is as follows:

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-\frac{2\pi jnK}{N}}$$

where N is the number of samples indexed by k in $x(n)$ discretized signal (Abdul & Al-Talabani, 2022).

4.4.4 Mel Filter Bank

To mimic the human ear's frequency sensitivity, the power spectrum is then filtered using a Mel filter bank, a set of overlapping triangular filters spaced according to the Mel scale. This scale approximates human auditory perception by spacing filters closer together at lower frequencies and farther apart at higher frequencies. The conversion from frequency f to Mel scale m is achieved by

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

with an inverse of this formula available to revert from Mel to frequency as needed. This Mel scaling step ensures that the resulting coefficients are more aligned with human auditory sensitivity, particularly beneficial in speech processing.

Mel frequency bands are equally spaced and extracted by triangular filters, these filters make up the filter bank (Abdul & Al-Talabani, 2022), (Inkeaw, 2023). The transfer function for mel band m is:

$$H_m(k) = \begin{cases} 0 & k < f(m-1) \\ \frac{k - f(m-1)}{f(m) - f(m-1)} & f(m-1) \leq k < f(m) \\ 1 & k = f(m) \\ \frac{f(m+1) - k}{f(m+1) - f(m)} & f(m) < k \leq f(m+1) \\ 0 & k > f(m+1) \end{cases}$$

where $f(m)$ is the central frequency of the triangular filter and

$$\sum_m^{M-1} H_m(k) = 1$$

Mel bands are an important parameter of the feature extraction part of the code, as different settings can affect the quality of the features extracted and ultimately the model performance.

After Mel filtering, the logarithmic compression step is applied to the filtered spectrum, where each Mel frequency component's amplitude is transformed to a logarithmic scale. This non-linear transformation simulates the human ear's response to volume changes, reducing the range of values and enhancing low-amplitude components, which makes it easier to detect softer sounds that may contain critical information. (Abdul & Al-Talabani, 2022)

4.4.5 Discrete Cosine Transform

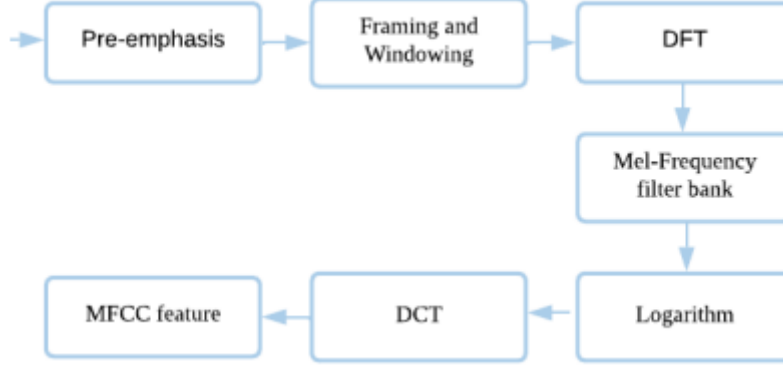
The final step is applying the Discrete Cosine Transform (DCT) to the log Mel spectrum, producing the MFCCs. The DCT is valuable because it compacts the energy into the first few coefficients, allowing for dimensionality reduction while retaining essential information. The formula for the DCT is:

$$X(k) = \sum_{n=0}^{N-1} x_n \times \cos\left(\frac{2\pi jnK}{N}\right)$$

Typically, only the first few MFCCs (X) are kept, as they capture the bulk of the perceptual features, while higher-order coefficients tend to capture noise (Abdul & Al-Talabani, 2022). In the

current scenario, it was originally decided to use 13 coefficients, which seems to be common in the audio processing industry.

Figure 4: MFCC framework



Source: (Abdul & Al-Talabani, 2022)

4.5 Energy

The authors of the original paper have found that including the energy of the signal as a feature proved useful as well (Aneja & Li, 2019). This energy is calculated via the Root-Mean-Square (RMS) of the signal and then taking the log of the result. Here, since the signals are discretized, the energy can be calculated as:

$$x_{RMS} = \sqrt{\frac{1}{n}(x_1^2 + x_2^2 + \dots + x_n^2)}$$

where n is the length of the signal (Wikimedia Foundation, 2024).

4.6 Counteracting large audio changes

Due to the nature of viseme transitions, there is a correlation between the switch from one viseme to another and significant audio changes. To reduce noise and increase consistency, the derivatives of the MFCC and energy values are included in the feature vector. (Aneja & Li, 2019)

Sampling happens at 100Hz, which in turn can produce noisy derivatives. To solve this problem, the derivatives are calculated on 5 frames (2 before and 2 after the actual frame), taking the average of these changes as the value for the actual frame. This introduces a larger latency but allows for

smoother derivatives. In addition, this process includes some lookahead, which is helpful in this application (Aneja & Li, 2019). The energy values go through the same process.

4.7 Librosa

“librosa is a python package for music and audio analysis. It provides the building blocks necessary to create music information retrieval systems.” (McFee, 2024)

Librosa is a Python library specialized in audio and music analysis, offering a suite of tools for audio feature extraction, processing, and visualization. It provides efficient implementations for tasks such as time-domain and frequency-domain analysis, including functions to compute mel spectrograms, chroma features, and spectral contrast, among others. Librosa's API includes functionality for loading audio files, handling sample rates, and supporting various transformations and representations, such as Discrete Fourier Transform (DFT) and Discrete Cosine Transform (DCT). Its integration with NumPy enables high-performance operations on large audio datasets, making it suitable for preprocessing steps in machine learning pipelines, particularly in speech and music classification tasks. (McFee, 2024)

There are some settings related to the sound that need to be set to provide the necessary parameters to librosa functions and to set up the input stream. The input is handled by a library called PyAudio, which enables the code to receive an audio stream from an input device, in this case, a microphone.

The **FORMAT** variable defines the data format for audio streams. In this case, it is set to **pyaudio.paFloat32**, which indicates that the audio data will be handled as 32-bit floating-point values. This format is commonly used for high-precision sound processing and minimizes rounding errors during transformations.

The **CHANNELS** variable specifies the number of audio channels. Here, it is set to 1, meaning the audio is processed as mono (single channel). Mono audio is computationally cheaper and sufficient for most speech and viseme processing tasks.

The **RATE** variable defines the sampling rate of the audio stream, set to 16,000 Hz (16 kHz). This rate is common in speech processing and provides a good balance between audio quality and computational efficiency, as it captures sufficient detail for human speech.

The `CHUNK_DURATION` variable represents the duration of an audio chunk in seconds, set to 0.025 seconds (25ms). This value is set to achieve the 25ms duration 10ms stride sliding window.

The `CHUNK_SIZE` is calculated as `int(RATE * CHUNK_DURATION)`, resulting in 400 samples per chunk. This value defines the number of audio samples in a single processing window. For example, with a 16 kHz sampling rate, 400 samples correspond to 25ms of audio.

The `STRIDE_DURATION` specifies the shift between consecutive chunks, set to 0.01 seconds (10ms). This overlapping of windows helps capture temporal continuity in the signal, especially important for speech dynamics.

The `STRIDE_SIZE` is computed as `int(RATE * STRIDE_DURATION)`, resulting in 160 samples. This determines how far to slide the analysis window for each subsequent frame.

The `p` variable initializes an instance of the `pyaudio.PyAudio` class, which serves as the interface for managing audio streams. This instance is used to record or process live audio from the microphone.

5 Feature Extraction

There are two ways I needed to extract features. One was from audio files, the other was from a live audio input stream. Therefore, I came up with two solutions with some key differences that allowed me to consistently extract audio features.

`extract_features_live:`

This function is designed to extract features from a short audio buffer, typically used in real-time applications like live audio processing from a microphone. The function begins by normalizing the audio buffer to ensure numerical stability. It converts the audio buffer to float32 format and scales it such that the maximum absolute value becomes 1. This normalization step prevents issues with numerical precision and ensures consistency in feature extraction. A hard limiter on the audio chunk is then applied during inference, right before this function call.

The function computes MFCCs which represent the power spectrum of the audio on a perceptual mel scale. Additionally, it calculates the first derivative (delta) of the MFCCs to capture temporal changes in the audio. The root mean square (RMS) energy is computed for each frame, which is then converted to a logarithmic scale for dynamic range compression. Similarly, the function computes the first derivative (delta) of the log energy to capture temporal variations in energy.

Finally, all the features—MFCCs, delta MFCCs, log energy, and delta log energy—are concatenated into a single matrix. However, instead of returning all the extracted features, the function transposes the matrix and returns only the first frame (`features.T[0]`), making it suitable for live, frame-by-frame audio processing.

`extract_features_from_file:`

This function is designed to extract features from an entire audio file, making it suitable for offline analysis or batch processing tasks. It begins by loading the audio file from the specified file path and resampling it to the desired sampling rate (100Hz in this case). Before extracting features, the function applies a hard limiter, which limits the audio amplitude to prevent distortion or clipping. After this, the audio buffer is converted to float32 format and scaled such that the maximum absolute value becomes 1, ensuring the data is normalized.

MFCCs are then calculated for the entire audio file. If the audio file contains more than one frame of MFCCs (`mfcc.shape[1] > 1`), the function computes the first derivative (delta) of the MFCCs to capture temporal changes, as well as the RMS energy for each frame. The RMS energy is then converted to a logarithmic scale, and its first derivative (delta log energy) is also calculated to capture temporal variations. All these features—MFCCs, delta MFCCs, log energy, and delta log energy—are collected into a list. However, if the audio file only produces a single frame of MFCCs, the delta and energy features are set to None.

The function returns a list containing all the extracted features for the entire audio file. This output makes the function suitable for offline processing where analyzing the entire duration of the audio is required.

Since the target frame rate is 100Hz or 100 FPS, the sampling rate is set to 16000 and the hop length is 160. Sampling rate is the number of audio samples captured per second in Hertz and hop length is the number of audio samples between the start of consecutive frames. The frame rate can be calculated as:

$$\text{Frame Rate} = \frac{\text{Sampling Rate}}{\text{Hop Length}} = \frac{16000}{160} = 100$$

The goal is to have a sample window with a duration of 25ms with a 10ms stride. The 10ms stride is set by the hop length like:

$$\text{Stride} = \frac{\text{Hop Length}}{\text{Sampling Rate}} = \frac{160}{16000} = 0.01s = 10ms$$

and `n_fft`, which is the number of samples in the Fast Fourier Transform window, ensures the 25ms duration, having a similar formula:

$$\text{Window Duration (seconds)} = \frac{n_fft}{\text{Sampling Rate}} = \frac{400}{16000} = 0.025s = 25ms$$

The differences between the two functions:

Table 2: File & Live extraction differences

Aspect	Live	From File
--------	------	-----------

Input	Takes a preloaded audio buffer (e.g., from a microphone).	Loads audio from a file
Hard Limiter	Normalizes the live audio buffer that has already gone through a hard limiter	Applies a hard limiter before normalizing the audio file.
Feature Scope	Extracts features for a short live audio buffer and returns the first frame only.	Extracts features for the entire audio file and returns all frames as a list.
Delta Features	Always computes delta MFCCs and delta log energy.	Only computes delta features if there is more than one frame in the MFCC matrix.
Output	Returns a single feature frame (the first one).	Returns a list containing MFCCs, delta MFCCs, log energy, and delta log energy for all frames (or None if unavailable).
Application	Designed for real-time processing.	Designed for offline processing of audio files.

Source: Self made

In the end, feature extraction produces files that consist of features extracted every 10 milliseconds from a 25ms window. A window represented in the JSON files looks like this:

```

{
  "start_timestamp_ms": 1980.0,
  "duration_ms": 25.0,
  "mfcc": [
    -349.0879211425781,
    65.16900634765625,
    -11.63652229309082,
    11.619379043579102,
    2.0114550590515137,
    -7.38970947265625,
    -1.9582092761993408,
    1.4860763549804688,
    0.3910481929779053,
    -2.1594645977020264,
    0.902534008026123,
    -6.911870002746582,
    -1.363754153251648
  ],
  "delta_mfcc": [
    -16.142471313476562,
    1.0749969482421875,
    5.977084636688232,
    -0.9111925363540649,
    -1.6980944871902466,
    0.40650129318237305,
    -0.4397122859954834,
    3.0577003955841064,
    -2.5424644947052,
    0.33946630358695984,
    0.879725456237793,
    0.7958962917327881,
    -0.5634075403213501
  ],
  "log_energy": [
    -6.350471019744873
  ],
  "delta_log_energy": [
    -0.20234107971191406
  ]
}

```

5.1 Combining the data

The next step is to assign each viseme to the correct audio feature vector based on the timestamps.

The process is as follows: first, the function `pair_mouthcues_with_features` opens and reads two JSON files: one containing a list of audio features (`features_file`), such as MFCC (Mel-frequency cepstral coefficients), delta MFCC, log energy, and delta log energy, and another containing a set

of visemes (mouthcues_file), which are visual representations of phonemes. These visemes are stored in the "mouthCues" key within the JSON structure.

The pair_mouthcues_with_features function is designed to align audio features, such as MFCCs and log energy, with mouth cues, which visemes. It takes three input arguments: the file containing audio features (features_file), the file containing mouth cues (mouthcues_file), and the path for the output file (output_file). The function reads the features and mouth cues from their respective JSON files, pairs them based on overlapping time intervals, and saves the paired data into the output file.

The function begins by loading the audio features and mouth cues from their respective files. The audio features include a list of entries, each containing timestamps (start_timestamp_ms and duration_ms) and associated feature data like MFCCs and log energy. The mouth cues include a list of intervals (start and end in seconds) along with their corresponding visemes. To align the time formats, the function converts the mouth cue timestamps from seconds to milliseconds by multiplying them by 1000.

For each audio feature, the function calculates its start and end timestamps in milliseconds. It then iterates through all mouth cues to identify those that overlap with the feature's time interval. The overlap condition is defined such that a mouth cue's start time is earlier than the feature's end time, and its end time is later than the feature's start time. If this condition is met, the mouth cue is considered relevant and added to a list.

Once all relevant mouth cues are identified for a feature, the function creates a dictionary containing the feature's timestamp, duration, associated mouth cues, and the extracted audio features (mfcc, delta_mfcc, log_energy, delta_log_energy). This dictionary is appended to a list that stores the paired data for all features. The function repeats this process for every feature in the input data.

Finally, the function saves the paired data into the specified output file in JSON format. Each entry in the output file contains an audio feature segment with its associated mouth cues and audio feature data. This alignment is crucial for synchronizing audio features with visual animations.

6 Data Loading

The combined data JSONs are the main data source that the code processes. They first get read then transformed into feature vectors in the form of tensors that PyTorch can work with. Naturally the training process utilizes the dataset, while during inference, an audio stream via microphone get transformed into vectors.

6.1 Data Loading During Training

Since the data source is separated into different files and I wanted to process each sentence separately, I opted for a generator design for the data loader function. Generators are functions that only process a small part of the data at one time, waiting until the next iteration is ready to be executed. In Python, this can be done with the `yield` keyword in place of the final `return` keyword that turns the function into a generator (Schemenauer, Peters, & Hetland, 2001).

In the code, it is called `data_generator` and it processes the JSON files in a specified folder, transforms the data into sequences of features and labels, and yields batches of tensors. The data is also normalized before being processed.

Normalization is crucial in machine learning and deep learning tasks for several reasons, particularly when working with input features such as audio signals, images, or numerical data. In this case, audio features such as MFCCs and log energy are derived from different processing methods and can have varying magnitudes. Without normalization, features with higher numerical ranges (e.g., log energy) might dominate the model's training process, making it harder for the model to learn from less dominant features like MFCC deltas. By normalizing each feature type (e.g., subtracting the mean and dividing by the standard deviation), all features contribute equally, improving the model's ability to learn meaningful patterns.

Normalization scales the input features to have similar ranges (e.g., between 0 and 1 or with zero mean and unit variance). This prevents features with larger magnitudes from dominating the gradient calculations, which can cause instability or slow convergence in optimization algorithms like gradient descent. For instance, if one feature has values in the range $[0, 1000]$ and another in $[0, 1]$, the larger-scaled feature will disproportionately influence the model's updates unless normalized.

Mathematically, normalization transforms data using:

$$x_{normalized} = \frac{x - \mu}{\sigma}$$

Where:

μ is the mean of the feature, and

σ is the standard deviation of the feature

In the code, the standard deviation and mean of every feature is precalculated on the actual dataset, then a normalization function is called on each individual JSON file's features when it is being loaded by the data generator.

Input Parameters

folder_path: Path to the folder containing JSON files with data.

label_encoder: A pre-fitted LabelEncoder object used to encode categorical labels into numerical values.

frames_per_sequence: The number of frames included in each sequence of data.

batch_size: Number of sequences in each batch.

steps_per_epoch: The maximum number of batches to yield per epoch (optional).

stats: Normalization values required for processing the data.

Workflow

1. **Initialize variables:** The function begins by listing all JSON files in the folder. The `file_names` list stores these filenames. Empty lists `batch_features` and `batch_labels` are used to accumulate data for batching, and `steps` tracks the number of batches yielded.
2. **Infinite Loop for Data Generation:** The function enters an infinite loop to continuously shuffle the `file_names` list (using `np.random.shuffle`) and iterate through the files. This ensures that data is presented in a different order in every epoch.
3. **Load and Process Each File:** Each file is loaded, converted to a Pandas DataFrame via `json_to_dataframe`, and filtered for non-empty data. The data is also normalized in this

step. The `label_encoder` encodes the labels, while features are extracted by dropping the label column. All numerical values are converted using `pd.to_numeric`.

4. **Divide Data into Sequences:** The data is split into full sequences of length `frames_per_sequence` using integer division. The remainder is handled separately by padding if necessary.

4.1. For each sequence, features and corresponding labels are sliced from the dataset, then they are appended to `batch_features` and `batch_labels` lists.

4.2. If the data length is not an exact multiple of `frames_per_sequence`, the last portion is padded with zeros to reach the required length. This is not a problem, since zero feature vectors are assigned X labels which correspond to silence.

5. **Batch Yielding:** Once the accumulated features and labels reach the `batch_size`, the function yields them as PyTorch tensors. These tensors are converted to float (`torch.float32`) for features and long integers (`torch.long`) for labels. After yielding a batch, the lists are cleared.

6. **Steps Per Epoch:** If `steps_per_epoch` is provided, the generator stops yielding batches after the specified number of steps. This ensures consistency in the number of iterations per epoch.

7. **Final Partial Batch Handling:** If some data remains in `batch_features` and `batch_labels` after processing all files, it is yielded as the final batch before clearing the lists.

The function outputs a tuple of PyTorch tensors:

- **Features Tensor:** `[batch_size, frames_per_sequence, num_features]`.
- **Labels Tensor:** `[batch_size, frames_per_sequence]`.

This solution ensures that the data is being fed to the model in equal size batches with equal size sequences, despite the fact that each recording has a different duration.

6.2 Data Loading During Inference

Naturally, the real-time audio processing step during inference does not include pre-stored data. Instead, a microphone provides an input stream that is then processed to extract the features from it and feed it to the model to make predictions.

1. **Audio Capture:** Audio data is captured in real-time from a selected audio source using the pyaudio library. The input is initialized via the `initialize_stream` function, which configures the audio stream based on parameters like sampling rate (`RATE`), format (`pyaudio.paFloat32`), and buffer size (`STRIDE_SIZE`). The audio stream continuously reads small chunks of audio (of size `STRIDE_SIZE`) and converts them into a NumPy array for further processing.
2. **Noise Reduction & Silence Filtering:** To enhance audio quality, a noise profile is optionally estimated using the `estimate_noise_profile` function, which collects short-duration audio samples. When processing each chunk of audio, noise reduction is applied using the Wiener filter. If no noise profile is available, the chunk is used directly. Each audio chunk's volume level is evaluated to determine whether it contains meaningful sound or silence. This is achieved using the Short-Time Fourier Transform (STFT) and converting the magnitude to decibels. If the volume is below the threshold (`db_threshold`), the system interprets it as silence, and an "X" viseme (representing silence) is emitted. Otherwise, the chunk is marked as containing sound, and further processing is applied.
3. **Feature Extraction:** For sound chunks, a hard limiter is applied using the `hard_limiter` function to normalize the audio intensity. Features are then extracted from the audio using `extract_features_live`.

After these steps, the prediction process begins. This pipeline ensures that raw audio is efficiently processed in real-time to produce reliable predictions for visemes.

7 The model

The model architecture is a standard unidirectional single-layer LSTM with a 200-dimensional hidden layer with 9 output nodes for each viseme. It takes a live sequence of feature vectors and outputs a sequence of viseme predictions.

7.1 Recurrent Neural Networks

What sets Recurrent Neural Networks (RNN) apart from other types of neural networks is the fact that they possess what we call a “memory”. This memory allows them to retain information from previous inputs to influence the current input and output, essentially “remembering” features that would be lost in other types of neural networks. When we use traditional neural networks, we assume that the inputs and outputs are independent of each other. In the case of the model described in this paper, since the data is sequential, the opposite is true. Each prior input essentially influences the prediction since there are common patterns in speech that help increase the accuracy of the output (IBM, 2024). Bidirectional RNN-s are also capable of taking into account future outputs, however, in the case of this application, the use of that architecture would likely lead to latency issues (Aneja & Li, 2019).

The input this model takes consists of audio features extracted from short audio recordings of human speech in the form of sentences. The output is a viseme and each word we speak consists of a series of sounds we make, which in turn is visible via the changes in the shapes of our mouth as we pronounce each word. This is why the data is described as sequential, since each previous sound influences the transition into the next one. However, there’s another sequence implicit in this data, which is the order of words in the sentences (this also makes the language of the training data a factor). These two sequences create the context, which is tracked by the RNN by maintaining a hidden state at every time step. This hidden state is the memory which allows the network to keep track of previous inputs. This hidden state is passed from step to step, creating a feedback loop, it is then processed alongside the new input, effectively using the memory built from previous inputs. (IBM, 2024)

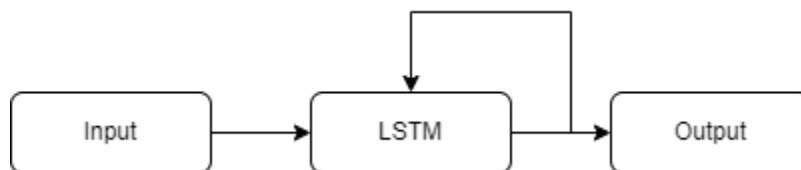
Another deviation from traditional neural networks is the fact that RNN-s use the one weight parameter per layer. These layers are then adjusted with a Backpropagation Through Time (BPTT) algorithm, which is used when working with sequence data. The idea of BPTT is largely the same

as regular backpropagation, with one key difference being that BPTT sums the errors at each time step (IBM, 2024). The way it achieves that is by “unrolling” the network at each time step. Every time step has an input timestep, a copy of the network which all share the same parameters, and an output. After unrolling, it calculates the error at each time step, then rolls the network up and sums all the weight updates together, this sum is then the actual update to the compact network (Werbos, 1990).

Today, many applications of RNN-s are outclassed by transformer models due to the vanishing and exploding gradient problems inherent to their structure and the fact that transformers possess longer-term memory and perform better at similar tasks (IBM, 2024). However, RNN-s are less resource intense and are still adequate for specific tasks. I believe it is prudent to minimize compute costs wherever possible, in order to use less energy and help preserve the environment while being more cost-effective at the same time.

7.2 Long Short Term Memory

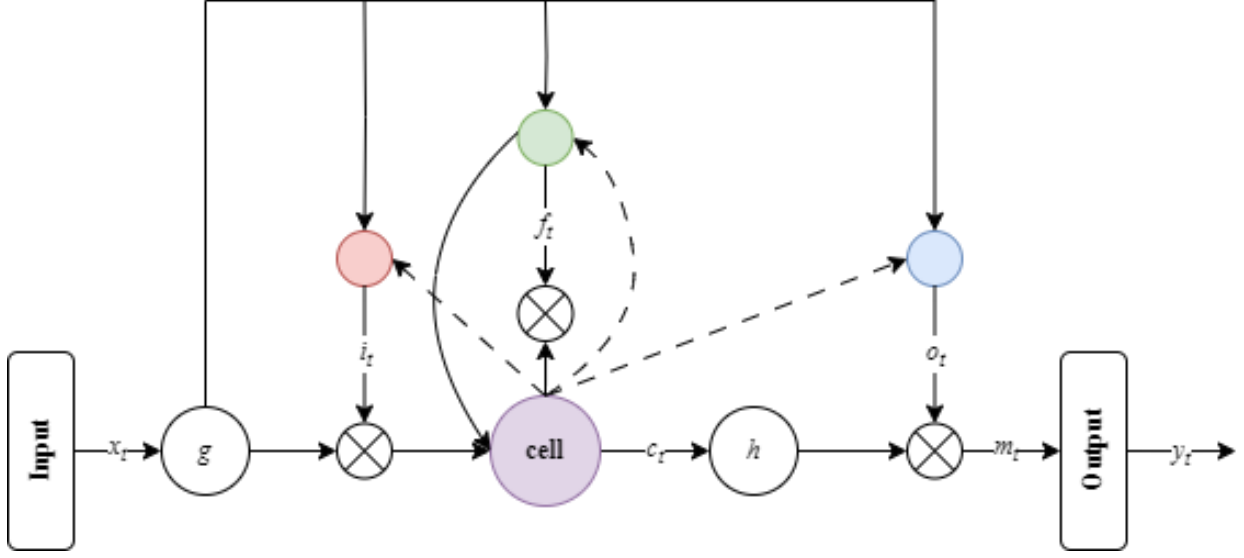
Figure 5: High-level overview of LSTM architecture



Source: Self made

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) designed to overcome issues with traditional RNNs in retaining information over long time intervals. In conventional RNNs, gradients can either vanish or explode over extended sequences, causing performance degradation. LSTM addresses this by introducing a mechanism to maintain a constant error flow within each memory cell through what is known as the "constant error carousel" (CEC). This structure allows LSTM units to store information for long periods without the gradient decay seen in traditional backpropagation through time (BPTT) methods.

Figure 6: LSTM Memory Block



Source: Based on (Sak, Senior, & Beaufays)

Each LSTM memory cell comes with gating units that control information flow, specifically the input (i), output (o), and forget gates (f), which selectively allow information to enter, remain, or exit the cell. The cell state is updated by adding the product of the input gate i_t , also scaling the previous cell state by the forget gate f_t , resetting or forgetting the block's memory and applying the cell activation vector (c) (Sak, Senior, & Beaufays). In addition, the LSTM architecture contains peephole connections from its internal cells to the gates in the same block to be able to learn the precise timing of the outputs (Sak, Senior, & Beaufays). The input sequence consists of the elements of $\mathbf{x}$ and the output sequence results in the elements of $\mathbf{y}$. The number of iterations is $\mathbf{t}$, so the calculations go as follows:

$$i_t = \sigma(W_{ix}x_t + W_{im}m_{t-1} + W_{ic}c_{t-1} + b_i)$$

$$f_t = \sigma(W_{fx}x_t + W_{fm}m_{t-1} + W_{fc}c_{t-1} + b_f)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g(W_{cx}x_t + W_{cm}m_{t-1} + b_c)$$

$$o_t = \sigma(W_{ox}x_t + W_{om}m_{t-1} + b_o)$$

$$m_t = o_t \odot h(c_t)$$

$$y_t = \Phi(W_{ym}m_t + b_y)$$

where:

- W refers to weight matrices in general,
- b refers to the bias vectors,
- W_{ic}, W_{fc}, W_{oc} are diagonal weight matrices for peephole connections,
- σ is the sigmoid activation function,
- m is the cell output activation vector
- $\odot$ is the operator for vector multiplication (elementwise),
- g is the cell input activation function ($\tanh$),
- h is the cell output activation function ($\tanh$),
- Φ is the network-level activation function, which is generally SoftMax.

To regulate how much of the stored memory should influence the next layer, the output gate o_t controls the cell output h_t . This is computed as the cell state passed through a nonlinearity, typically a hyperbolic tangent, and then modulated by o_t : $h_t = o_t \times g(c_t)$. This gated mechanism enables LSTMs to retain, update, or discard information efficiently over multiple time steps, facilitating learning in complex sequence tasks where relevant information may be far apart.

Pictured above [Figure 6] is just one memory block. LSTMs generally involve multiple memory blocks one-after-another.

In terms of computational efficiency, LSTM's complexity per time step and weight is $O(1)$, matching BPTT while offering advantages over real-time recurrent learning (RTRL) and other RNNs. The use of gated cells and the CEC allows LSTM to handle long time-lagged dependencies better than traditional methods, making it suitable for tasks involving noisy inputs, variable sequence lengths, and distant temporal dependencies. Unlike fixed state models, LSTMs can generalize well to problems where the position of inputs matters less than their occurrence, accommodating continuous data without requiring predefined state numbers. Consequently, LSTM has proven useful across applications such as time series forecasting, language modeling, speech recognition, and music composition, where long-term dependencies are vital (Sepp & Jürgen, 1997).

7.3 Loss Function

“A loss function is a type of objective function, which in the context of data science refers to any function whose minimization or maximization represents the objective of model training.” (IBM, 2024)

Also known as “cost function” or “error function”, I refer to it as “loss” or “error” interchangeably. The meaning of these two words reveals that the goal here is to minimize these functions in order to maximize prediction accuracy.

Loss is the difference between the predicted value and the actual value, the “ground truth”. The smaller the difference, the lower the loss, therefore the prediction accuracy is higher. Knowing the loss incurred during predictions is not only important to determine the accuracy of a model, but loss is also used in supervised learning, during the training phase in backpropagation, this is what indicates which way the model should adjust its weights and biases via gradient descent. (IBM, 2024)

The model described in this paper is a supervised learning model, meaning the dataset has labels, which, in this case, are the visemes assigned to audio feature vectors by the rhubarb lip-sync process.

7.4 Cross Entropy Loss

Cross Entropy Loss, also known as Log Loss, is a loss function commonly used to measure the performance of classification models with probability value outputs between 1 and 0. While the target probability of a prediction is better the closer it is to 1, we want the loss to be as close to 0 as possible, indicating an accurate prediction.

This loss function utilizes Cross-entropy (H) which is defined over p target distribution and q , which is the approximation of the target distribution as:

$$H(p, q) = -E_p[\log q]$$

where E_p is the expected value operator with respect to p , the target distribution. In my use case, the distribution is discrete, so Cross-entropy can be calculated as:

$$H(p, q) = - \sum_{x \in X} p(x) \times \log q(x)$$

where X is the support for both p and q , meaning the set x is part of (Brownlee, 2022), (Wikimedia Foundation, 2024).

For machine learning applications Cross-entropy is used as a loss function to calculate loss L , a different notation is used and the number of classes C is also included:

$$L = \sum_{i=1}^C y_i \times \log(\hat{y}_i)$$

where y_i is the label, and $\hat{y}_i$ is the predicted label for probability class i (Hastie, Tibshirani, & Friedman, 2009).

After training the model on a dataset with N samples, we get the complete loss of the model which is averaged over all samples as:

$$L = -\frac{1}{N} \sum_{n=1}^N \sum_{i=1}^C y_i^{(n)} \times \log(\hat{y}_i^{(n)})$$

where n represents the current sample, so both the true label and the predicted label are with respect to the specific sample (Hastie, Tibshirani, & Friedman, 2009). The result of this calculation provides a good indicator of how accurate the model is, so during training it is best to run it multiple times due to the stochastic nature of neural networks.

7.5 The PyTorch framework

“PyTorch is an optimized tensor library for deep learning using GPUs and CPUs.” (PyTorch Foundation, 2023)

PyTorch is an open-source deep learning framework that provides a flexible and efficient platform for building neural networks, making it particularly suitable for creating and deploying LSTM models. In PyTorch, LSTM (Long Short-Term Memory) layers are implemented through the `torch.nn.LSTM` module, which handles both the internal states and sequence of computations necessary for processing sequential data. PyTorch’s dynamic computation graph, or "define-by-run" architecture, allows modifications to the structure of the network during runtime, making it ideal for handling complex, variable-length sequences, as required by many LSTM applications.

Furthermore, PyTorch provides GPU support, which is essential for training computationally intensive LSTM models efficiently, particularly on large datasets.

When designing an LSTM model in PyTorch, developers define the network's structure using sequential layers, which may include one or more LSTM layers, fully connected layers, and activation functions, depending on the task requirements. PyTorch's data handling capabilities, through modules like `DataLoader`, simplify batching and shuffling of sequences for training, while `torch.optim` and `torch.nn` modules offer a range of optimization and loss functions to streamline training procedures.

7.6 Model structure

```
class LipsyncModel(nn.Module):  
    def __init__(self, input_size, num_visemes, label_mapping,  
                  dropout_ratio=0.5):  
        super(LipsyncModel, self).__init__()  
        self.lstm = nn.LSTM(input_size, 200, batch_first=True)  
        self.dropout = nn.Dropout(dropout_ratio)  
        self.fc = nn.Linear(200, num_visemes)  
        self.decoder = DecodingLayer(label_mapping)
```

LSTM Layer:

The LSTM layer captures temporal dependencies within sequences, which is useful for tasks where previous inputs influence current outputs, like lip-sync.

Parameters:

- `input_size`: The dimensionality of the input feature vector, which includes extracted audio features.
- `hidden_size=200`: The LSTM has 200 in its hidden state.
- `batch_first=True`: This ensures the input and output tensors are shaped with the batch dimension as the first axis, followed by the sequence length, and then the feature size.

Output: The LSTM layer outputs a sequence of hidden states, with each hidden state representing the information learned up to that point in the sequence.

Dropout layer: Dropout is a regularization technique that helps prevent overfitting by randomly setting a fraction of the layer's activations to zero during training.

Parameters:

- `dropout_ratio=0.5`: This specifies that 50% of the LSTM outputs will be randomly set to zero.

Effect: During training, dropout helps the model generalize better by preventing reliance on any specific neurons. During inference, dropout is inactive, and all neurons contribute to the output.

Fully connected (Linear) layer: This layer maps the output of the LSTM to the desired number of viseme classes, creating the logits for each viseme.

Parameters:

- `in_features=200`: The input size of the linear layer matches the LSTM's hidden size, taking in the 200-dimensional vector from the LSTM.
- `out_features=num_visemes`: The output dimension is set to `num_visemes`, which is the number of unique visemes (or mouth shapes) the model can predict.

Output: The linear layer produces a set of logits for each viseme. These logits represent the model's confidence in each viseme class.

Decoding layer: The decoding layer translates the model's output (logits) into human-readable labels (visemes) based on `label_mapping`.

Parameters:

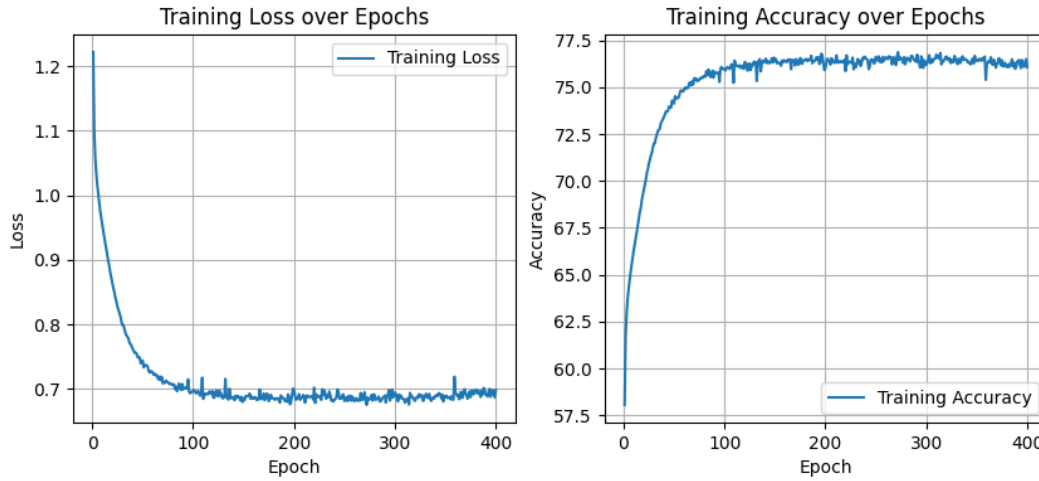
- `label_mapping`: A dictionary or mapping that converts class indices (integers) back into viseme labels.

Output: This layer decodes the integer predictions from the model into their corresponding labels, which can be directly used as the final viseme predictions.

7.7 Training

I used much of the same parameters as used in the original research. The difference is that I used data extracted from 6300 audio recordings, ten times as much as the paper this thesis is based on. I tested other parameter settings too but found that the optimal ones are already in the paper. For example, increasing the epochs from 200 to 400 didn't reveal convergence to a lower training loss value or to a higher training accuracy value [Figure 7].

Figure 7: Training loss & convergence over epochs



Source: Code generated

7.7.1 Baseline

The baseline accuracy for a classification model such as this with 9 output classes is 11.11%. For a model making random predictions across N classes, the probability of correctly predicting any given class is $\frac{1}{N}$. The cross-entropy loss for such a uniform distribution is calculated as:

$$-\log_e\left(\frac{1}{N}\right)$$

Therefore, with 9 output classes, the baseline loss is around 2.197.

The baseline describes the loss and accuracy when the model is just randomly guessing. Surpassing the baseline generally means that the model has picked up on patterns in the dataset.

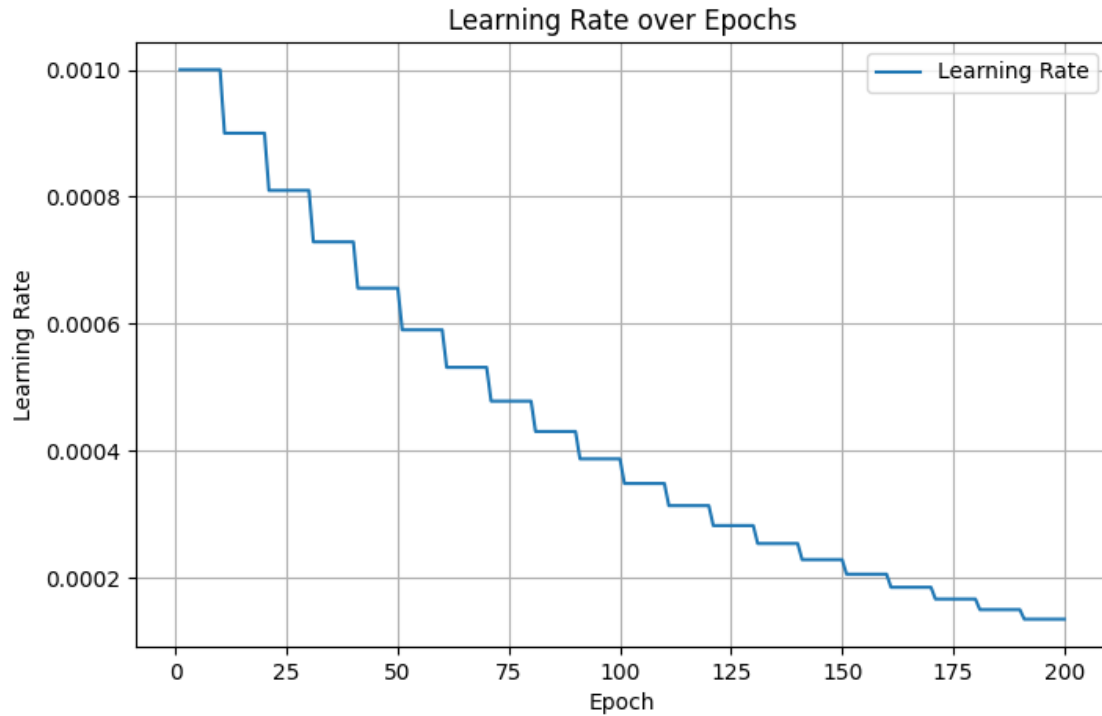
7.7.2 Experiments

I ran experiments with a setup, where instead of making predictions on audio-viseme sequence pairs, the model was set to make predictions on individual visemes in the 10ms intervals. This method did not work at all and resulted in extremely chattery transitions. Instead, I used 1000ms long sequences as it was one of the experiments showing good results in the original research (Aneja & Li, 2019) and this worked well.

The first attempt was done with a setup as similar to the paper as I could get it. 200 epochs, with a batch size of 20 and a learning rate of 0.01 and a vector sequence length of 1 second. This model wasn't very successful, so I decided to diagnose the problem by creating graphs during the training process.

In addition to training and testing, a common step in machine learning is validation. Validation is usually 10% of the training set separated for this purpose. Running a validation step after each epoch helps diagnose problems early. This is crucial when training large models, but even in this case it helps find issues, since, unlike testing, validation shows a tendency, rather than just pure results.

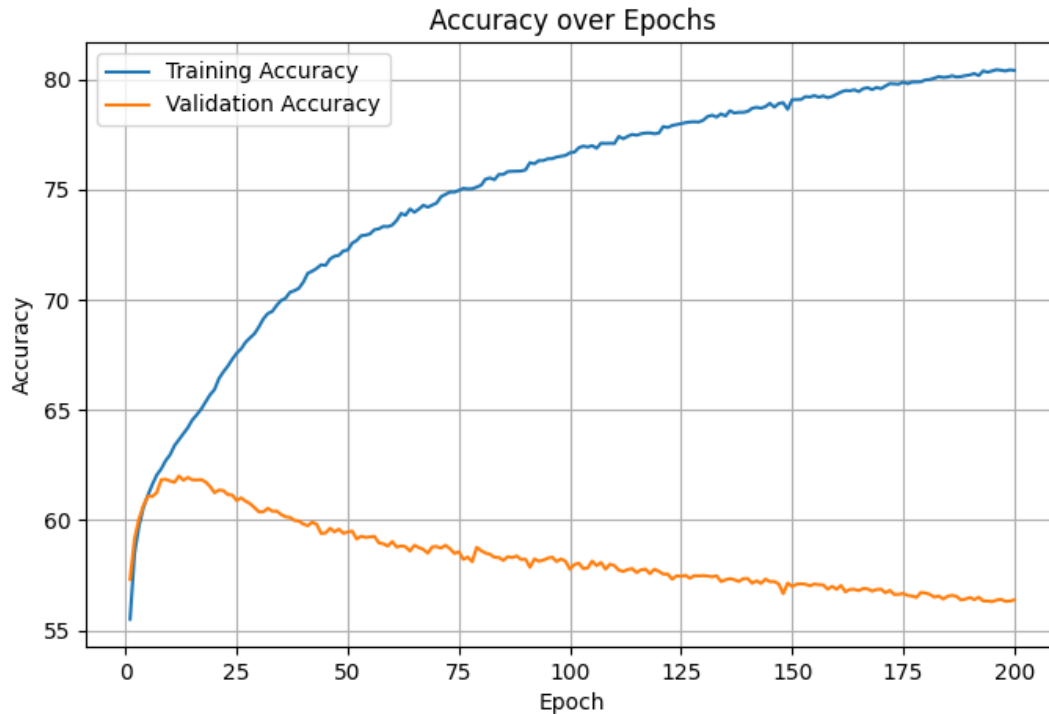
Figure 8: Learning rate change over epochs



Source: Code generated

Taking a look at the learning rate first [Figure 8], it looks typical, nothing is out of the ordinary since I am using a learning rate scheduler built into PyTorch, StepLR gradually lowers the learning rate to avoid overshooting during optimization.

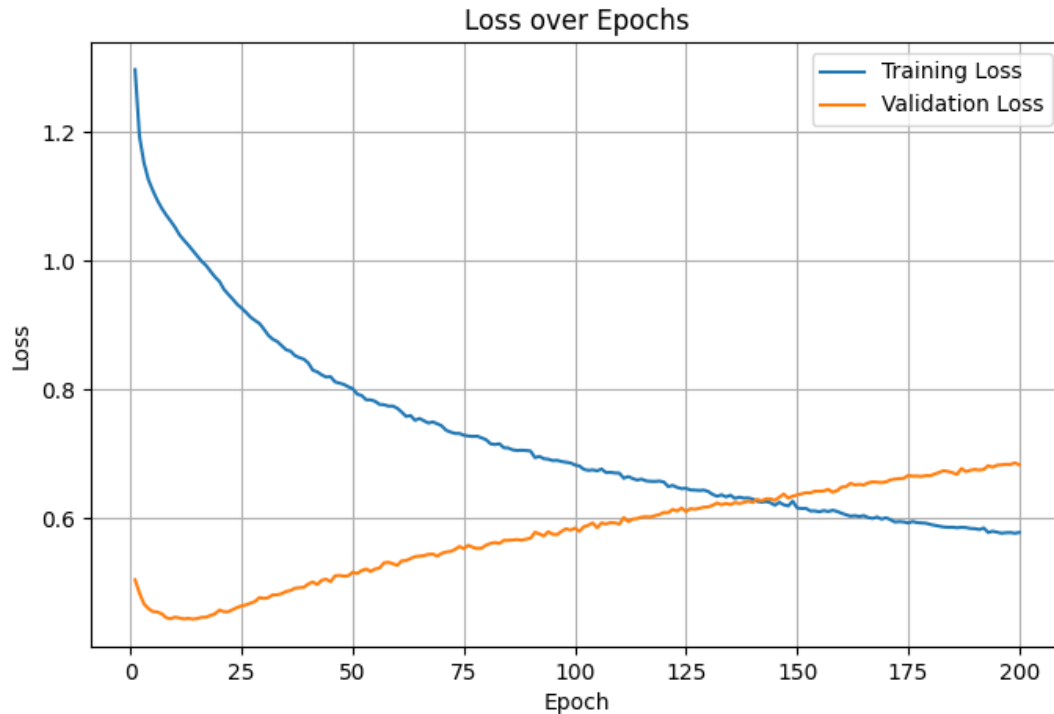
Figure 9: Training & Validation accuracy over epochs



Source: Code generated

Next, I examined the accuracy plots [Figure 9] and here I found the first evidence of heavy overfitting. This is evident by the large gap between training and validation accuracy scores. Training accuracy grows steadily to reach over 80 percent, however, validation spikes, then diverges from training. The spike signals that the model successfully learns some rules, however, the decline then means that a feature is overrepresented in the training set. Ideally, training and validation accuracy would both increase or at least remain stable with minimal divergence, indicating that the model is learning generalizable patterns. The consistent drop in validation accuracy here suggests that the model is failing to generalize beyond the training data.

Figure 10: Training & Validation loss over epochs



Source: Code generated

The loss plots [Figure 10] confirm the issue of overfitting seen in the accuracy graph. The training loss decreases consistently, which suggests that the model is learning from the training data and fitting it well. However, the validation loss initially decreases slightly but then starts to increase, even as training loss continues to decline. This is a classic sign of overfitting. The model is learning to fit the training data too closely and is not generalizing well to the validation data.

In a well-generalized model, both training and validation loss would ideally decrease and converge. Here, the divergence between the two suggests that the model is learning patterns specific to the training data but not transferable to new, unseen data.

Overfitting occurs when a machine learning model learns not only the underlying patterns in the training data but also the noise, outliers, and specific details that do not generalize to unseen data (Dietterich, 1995). This happens when the model becomes excessively complex relative to the amount of training data or lacks sufficient regularization, leading to excellent performance on the training set but poor performance on validation or test sets. Overfitting results in a model that is overly tailored to the training data, making it less capable of adapting to new inputs and often

failing to generalize to real-world scenarios. Techniques such as regularization, dropout, data augmentation, and early stopping are commonly used to mitigate overfitting.

To dig further into what is causing the issue, I also created a confusion matrix, to explore which classes get mistaken the most.

Figure 11: First experiment confusion matrix

		Confusion Matrix								
Actual	Class 0 -	3093	6150	1963	260	880	658	266	81	883
	Class 1 -	2562	70646	12447	1180	5468	4815	1079	805	4222
	Class 2 -	1155	18325	17629	1237	3046	1437	292	494	721
	Class 3 -	277	2504	2207	1968	1359	284	50	69	78
	Class 4 -	612	8136	3505	902	7957	1446	175	430	323
	Class 5 -	400	9188	1993	219	1559	4781	114	246	382
	Class 6 -	334	2648	500	53	354	247	1610	35	194
	Class 7 -	119	1722	886	114	696	352	29	1017	86
	Class 8 -	285	4121	502	50	267	296	187	26	28294
		Class 0	Class 1	Class 2	Class 3	Class 4	Class 5	Class 6	Class 7	Class 8
		Predicted								

Source: Code generated

This graph [Figure 11] reveals a significant amount of information about the model's performance.

High True Positive Counts for Major Classes:

- Class 1 and Class 8 have high true positive counts (70646 for Class 1 and 28294 for Class 8). This indicates that the model can correctly identify these classes with relatively high confidence compared to others.
- However, the model is heavily reliant on these two classes, and the high counts here could be due to class imbalance or the model overfitting to these classes, which it finds easier to learn.

Misclassification in Neighboring Classes:

- There is significant misclassification across several classes. For instance:

- Class 1 is often misclassified as Class 2 (12447 times) and Class 4 (8136 times), showing a tendency for the model to confuse these classes.
 - Class 2 has many instances misclassified as Class 1 (18325 times) and Class 4 (3505 times).
- This suggests an overlap in the features of these classes, making it difficult for the model to distinguish them accurately.

Inconsistent Recognition of Certain Classes:

- Classes 0, 3, 4, 5, 6, and 7 show relatively low true positive counts with many instances being classified as other classes, particularly Class 1 and Class 2.
- For example, Class 4 (often misclassified as Classes 1, 2, and 8) has very few true positives, indicating that the model struggles to identify it correctly.

Possible Class Imbalance:

- The high counts in certain classes (particularly Class 1 and Class 8) and lower counts in others suggest that there might be an imbalance in the dataset, where some classes have more examples than others.
- If this is the case, the model may have biased itself towards the more frequently occurring classes (like Class 1), leading to poorer performance on the less represented classes.

Model Generalization Issues:

- The spread of misclassifications across almost all classes indicates that the model might be struggling to generalize well. This aligns with the signs of overfitting observed in the previous accuracy and loss graphs.
- The model might have memorized certain patterns that allow it to perform well on specific classes (such as Class 1 and Class 8) but is less effective in generalizing across all classes.

Since the model uses all the classic methods to reduce overfitting like dropout, regularization, and correct parameters, a large enough and well distributed data source, the issue lies in the dataset. This assumption was also strengthened by the second experiment.

In the second experiment I added an additional LSTM layer to the model. Essentially attempting to brute-force a more complex set of rules. Since I was using more data than the original research, this was applicable, since more data can lead to more intricate generalizations. The additional layer was exactly the same as the first LSTM layer, I simply added more capacity to the model.

The results of the second experiment were largely similar to the first one. This meant that the model already had enough capacity to learn on this dataset, the issue is not in the model architecture.

In the original paper, the dataset was created from a subset of the TIMIT dataset (Aneja & Li, 2019). It is mentioned that they only used SX sentences, which the dataset creators described as “*phonetically compact*” (Garofolo, 1993). These sentences were based on prompts that provide a good coverage of pairs of phones with additional phonetic contexts that were deemed as difficult or interesting. Although I assumed this subset was selected to reduce the amount of sentences that needed to be manually animated, while keeping the maximum phone transition variety, after seeing the overfitting results, I decided to run an experiment only using SX sentences. In theory, only using a subset of the data is not ideal, however, since SX sentences make up 50% of the recordings (Garofolo, 1993), I thought the other 50% could potentially cause noise. SX sentences are also only spoken by 7 speakers, a massive reduction from the full 630. Still, this could potentially reduce noise if the model is quick at generalizing and so far, has been held back by the amount of noise.

This proved to be untrue, however, after running a third experiment. The loss and accuracy curves showed the same tendencies, their peaks were lower than they were on the full dataset. This showed that actually, using the full dataset did not introduce too much noise, despite the differences in subsets. It also meant that Rhubarb lip sync performed the same on all subsets, as there was no difference in the confusion matrix from before, meaning that the feature distribution remained the same.

Figure 12: Third experiment confusion matrix

		Confusion Matrix								
Actual	A	1576	3816	1306	182	688	291	150	60	467
	B	1623	28888	6637	506	2822	2410	456	425	1759
	C	758	8205	6663	553	1634	658	168	271	370
	D	226	1330	1339	497	759	128	22	47	55
	E	464	4681	2233	454	3264	770	99	307	227
	F	364	3858	925	107	827	1358	79	229	194
	G	125	1375	292	46	169	135	837	45	48
	H	55	796	451	36	426	150	35	237	38
	X	214	1677	228	20	129	126	101	37	13804
		A	B	C	D	E	F	G	H	X
		Predicted								

Source: Code generated

One improvement, however, was that I decoded the classes before displaying them on the confusion matrix [Figure 12] to be able to see which labels are causing the problems. I did expect ‘X’ to be an outlier as each recording begins and ends with a small amount of silence, it would be easy to generalize. To see that ‘B’ was the most recognizable revealed some information as well. ‘B’ is the viseme used for most consonants like “K”, “S”, “T”, and even the “EE” sound.

Since the confusion matrices hinted at the issue lying in the spectral characteristics of each viseme being perhaps too compressed or noisy to recognize, I opted to increase the mel bands from 40 to 80 during feature extraction and regenerated the dataset. I used 40 by default as it is conventional (Abdul & Al-Talabani, 2022). This seemed to bring improvements; however, it was almost negligible.

These experiments all explored different directions to try to find an improvement, without much success. It seems like the model achieves its peak performance quite early in the training cycle and then starts overfitting. Comparing to the model in the original research, my model has access to much more data, so it is entirely possible that the model learns to generalize quickly, and a 200-

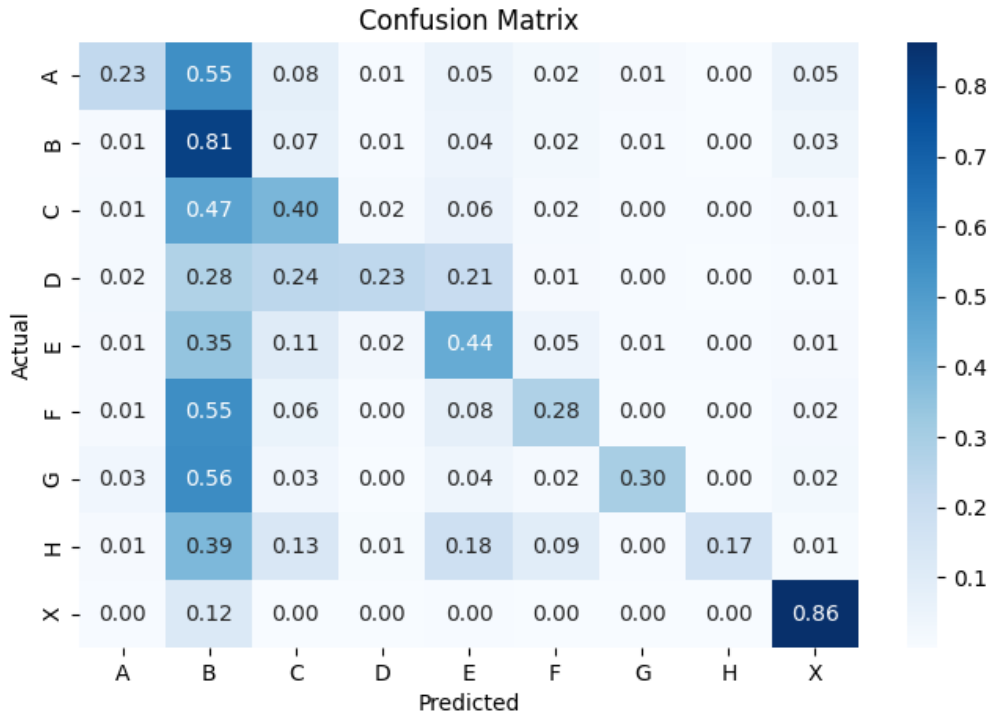
epoch training cycle is too much in this case, while it was necessary for the other one due to the amount of available data.

To test this theory, I implemented early stopping into the training process. Early stopping is a regularization technique used during model training to prevent overfitting and improve generalization. It monitors the performance of the model on a validation dataset at the end of each epoch and stops the training process when the validation performance stops improving for a specified number of consecutive epochs, known as the “patience” (Prechelt, 2000). The rationale is that when the validation loss no longer decreases or the validation accuracy plateaus, the model may begin to overfit the training data, learning patterns that do not generalize to unseen data. By stopping early, the model preserves the weights from the epoch with the best validation performance, effectively striking a balance between underfitting and overfitting. This technique also saves computational resources and time by avoiding unnecessary training after the optimal point.

The early stopping mechanism has a patience of 10 epochs, and if it does not see improvement during the patience period, it saves the model in its most accurate state. I used the 80 mel band dataset and the model reached its lowest validation loss at 12 epochs. The testing phase revealed a final accuracy of 59.65%. This may seem low in general terms; however, the original research reached an accuracy of 66.84% (Aneja & Li, 2019) only after post-prediction filtering. This was close enough to assume that the accuracy of my model after filtering will reach theirs.

In order to get a better understanding of what is happening, I improved the confusion matrix once again, by normalizing the values on the “Actual” axis. This way, it was much easier to see what was happening.

Figure 13: Fourth experiment confusion matrix



Source: Code generated

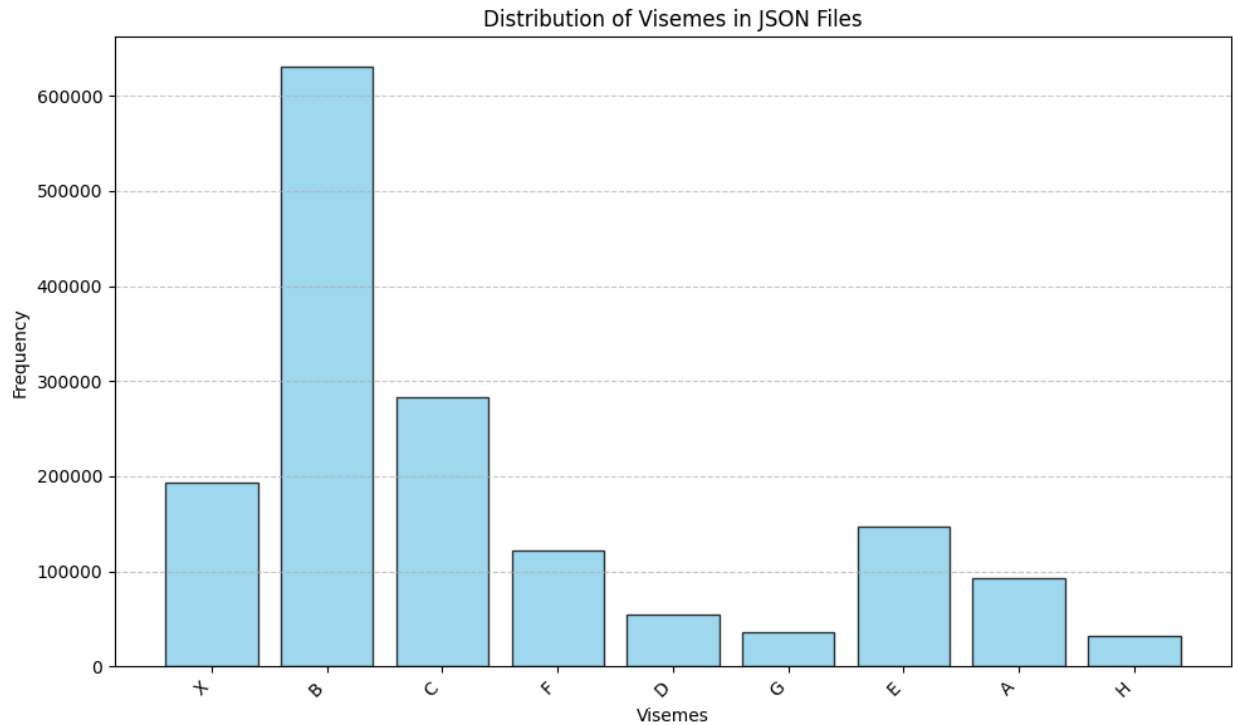
As can be seen in the graph [Figure 13], the ‘B’ viseme is causing problems. Every other viseme except silence has a significant chance of being mistaken for ‘B’, not only significant, but in many cases the actual viseme is predicted less than ‘B’.

This could be caused by two things:

- Overrepresentation
- Overly compressed audio

Overrepresentation means that one label is much more prevalent in the dataset than all others. In the case of ‘B’, this is possible because it is used for most consonants by Rhubarb lip sync (Wolf, 2022). Overrepresentation combined with noisy data can lead to the most common label being predicted instead of the actual ones. To verify this, I analyzed the training dataset, to see if there are any imbalances, by counting how many times each viseme is present in the data.

Figure 14: Distribution of visemes in the training dataset



Source: Code generated

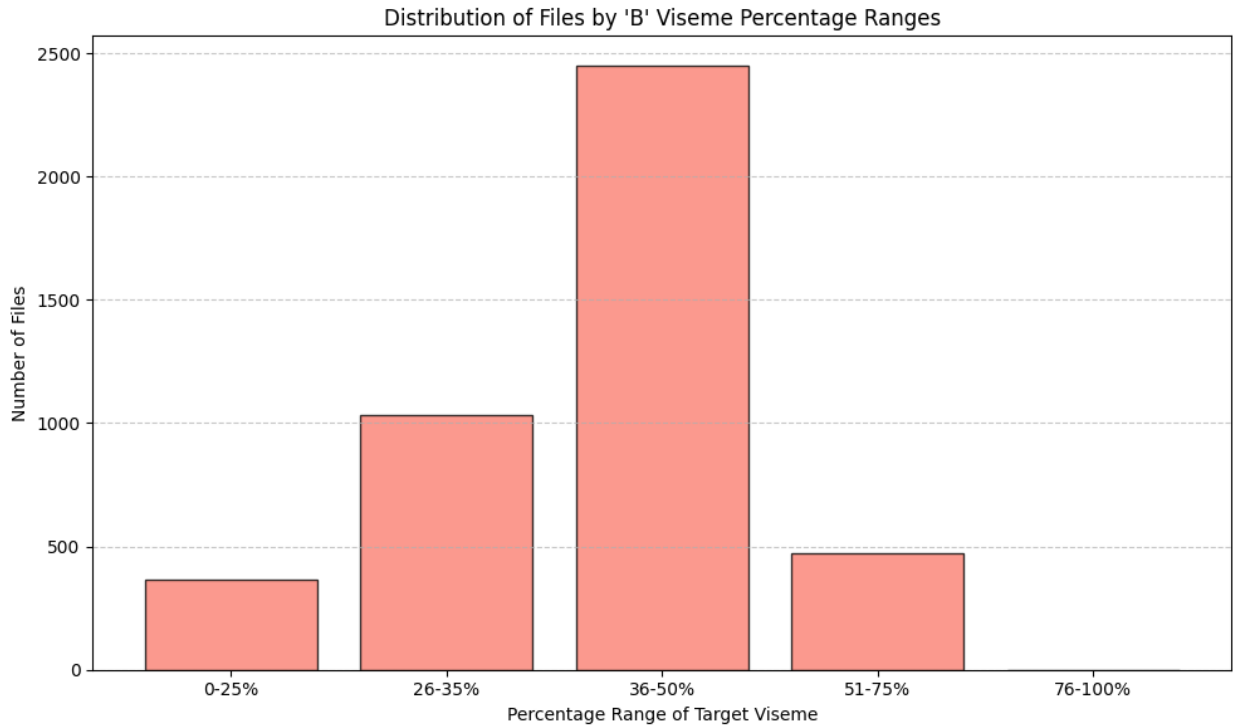
As visible on the graph [Figure 14], the ‘B’ viseme appears over twice as much as any other viseme. This explains why other visemes get confused for ‘B’, since if the data is noisy and the answer is unclear, statistically ‘B’ is often the right answer. The over 80% accuracy for ‘B’ is also possible because of how many examples of it the model is exposed to, compared to all the other visemes.

Seeing the results of the graph, it made sense to check if there are any sentences that are mostly made up of ‘B’ visemes, possibly erroneously labeled by rhubarb lip sync, or just being exposed to the software’s preference towards this particular viseme. To explore this, I graphed the distribution of files by their percentage of ‘B’ visemes [Figure 15], to see if I could eliminate sentences that have too high of a percentage of this viseme.

Unfortunately, the findings made here did not make it possible to eliminate a significant number of sentences. The largest range by number of files is the 36-50% range, being over twice as much as other ranges. The 51-75% range contains less than 500 files, so culling this range will not bring the desired effect while the range below it has nearly 2500 sentences. In addition, the graph shows that there are no anomalies in the labeling process by rhubarb lip sync, as the 76-100% range is empty.

Sentences in this range would almost certainly be mislabeled as it would be atypical of the English language.

Figure 15: Distribution of sentences by 'B' viseme content



Source: Code generated

One solution would be to separate some of the sounds that ‘B’ represents into different visemes. A “K”, “EE” and “S”, “T” split for example would at least increase the spread of the erroneous predictions, leading to a more believable animation. However, I do not have access to any other viseme set as I am using Rhubarb, this is a limitation for now, but during further development, more options can be explored.

To examine, how biased rhubarb lip sync is compared to human animators, I turned to the original research, where a graph [Figure 16] was made of the viseme preferences of each of the 4 animators. The idea was to see if they also preferred sounds represented by the ‘B’ viseme. Their viseme set is different, however, consisting of 12 different visemes instead of 9, but the sounds can be equated to this set as well.

Figure 16: Viseme distribution by frame for each animator



Source: (Aneja & Li, 2019)

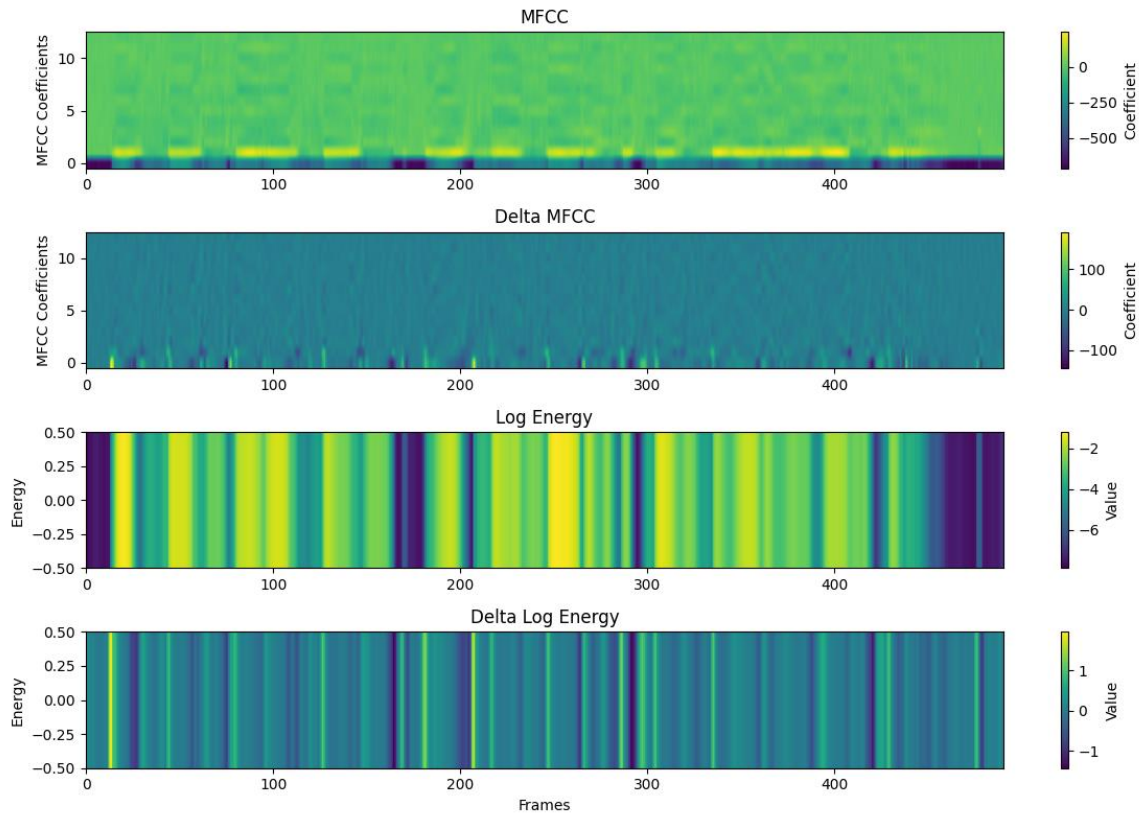
In the original research, the ‘E’ viseme represents the “EE” sound, and ‘S’ represents the “S” sound. ‘L’ and ‘D’ both represent “TH” which is close to “T” and ‘D’ also represents “G” which can be equated to “K”. Among these, ‘E’, ‘S’ and ‘D’ tend to be more popular visemes, albeit not always significantly more common than others. One factor to note is that these are based on human interpretation (Aneja & Li, 2019), not predefined rules and audio characteristics (Wolf, 2022), this means that an animator can choose a viseme that may not represent the actual sound, but it looks better to human perception to use that viseme instead of the “correct” one. Either way, the graph supports the idea that the ‘B’ viseme should be separated into at least 2 other visemes in rhubarb lip sync if it were to be used as the labeling software. I reached out to the creator, to ask why ‘B’ is such a multipurpose viseme, but as of the time of writing, I received no response.

The other possible explanation is that audio compression reduces the peaks too much and in turn, this multi-purpose viseme gets mistaken whenever another sound’s signal reaches the top of the spectrum. This is because the sounds “B” represents are all high-pitched and short, so they generally cause the peaks in the audio wave signal. Since I am using a hard limiter, these higher peaks can get reduced to the level of lower peaks, therefore causing confusion. However, the hard limiter is important and may not be excluded. During initial testing, I mistakenly forgot to apply the hard limiter while generating the dataset; this led to results significantly worse than the experiments made after fixing the mistake. The compression problem, however, is not solely responsible (if it is even significant) for the issue of visemes being mistaken for ‘B’. Overrepresentation is much more of a factor.

As another test, regenerated the dataset with 128 mel bands, to see if the added detail improves accuracy. The difference in accuracy from 80 mel bands to 128 was not significant. On most visemes this change meant 1% more accurate predictions, but the same tendency to overuse ‘B’ was still present.

After the unsuccessful attempts to improve the model’s accuracy, I decided to compare the features extracted from my microphone input and the features extracted from the waveform files in the dataset. I selected a recording, and used it as the input first, then I spoke the same sentence as in the recording into my microphone and compared the spectrograms.

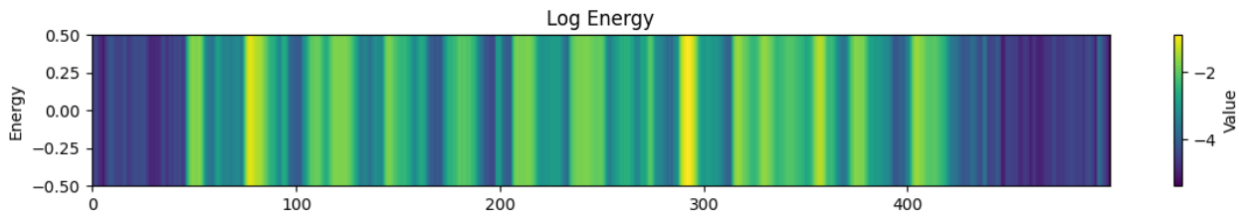
Figure 17: Feature spectrograms extracted from a recording in the TIMIT dataset



Source: Code generated

Above are the spectrograms of the features extracted directly from a selected recording from the TIMIT dataset [Figure 17]. Here, the peaks in log energy come across quite strong, with sharp changes, and the MFCC graph is also quite “full”.

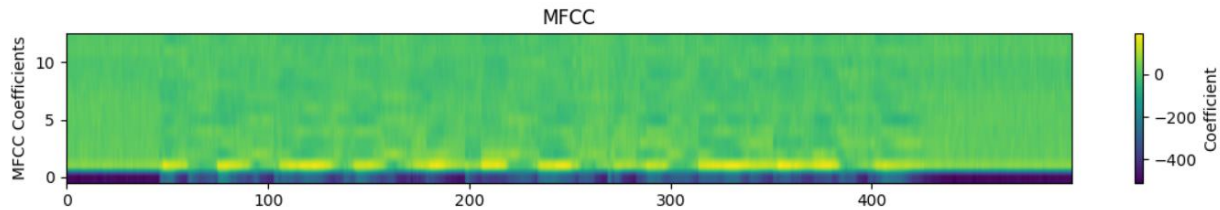
Figure 18: Log Energy spectrogram of my voice saying a sentence from the TIMIT dataset



Source: Code generated

The starkest difference is visible in the log energy spectrogram [Figure 18], as when I spoke the sentence, the log energy transitions were smoother and took longer, while also having lower peak values across the entire duration.

Figure 19: MFCC spectrogram of my voice saying a sentence from the TIMIT dataset



Source: Code generated

The MFCC spectrogram [Figure 19] shows lower values overall, meaning that my input is on a lower frequency on average.

Looking at the features, I noticed that the microphone quality at the time when the TIMIT dataset was recorded, was not on par with current microphones. Especially the “T” and “S” sounds have a “sizzling” effect to them, making them sound the same due to this high-frequency noise. This is important, because at this point, I was relying solely on the ability of Rhubarb lip sync to analyze the recordings. However, in the original research, the human animators did not rely simply on audio waves, they understood what the speakers were saying. Knowing the words in the audio recording is a major advantage, because even if the quality is bad, humans can create the correct viseme sequence regardless of that fact, having understood the words. Rhubarb lip sync has an option to read a text file that contains the transcript of the audio, and the TIMIT dataset includes transcript text files for each recording. I used this, to modify the code and during dataset generation, I passed on the location of the text file to Rhubarb in its command, to improve the viseme sequences. I trained the model with the new dataset.

Unfortunately, the results did not show a major improvement. On average, each class saw a 1-5% increase in the number of times it was identified, but the same pattern of ‘B’ being mistakenly predicted remained. This model would end up being the best one, with 59.71% accuracy on the test dataset and a slightly lower chance of predicting ‘B’ for any given input.

7.7.3 Ground Truth Testing

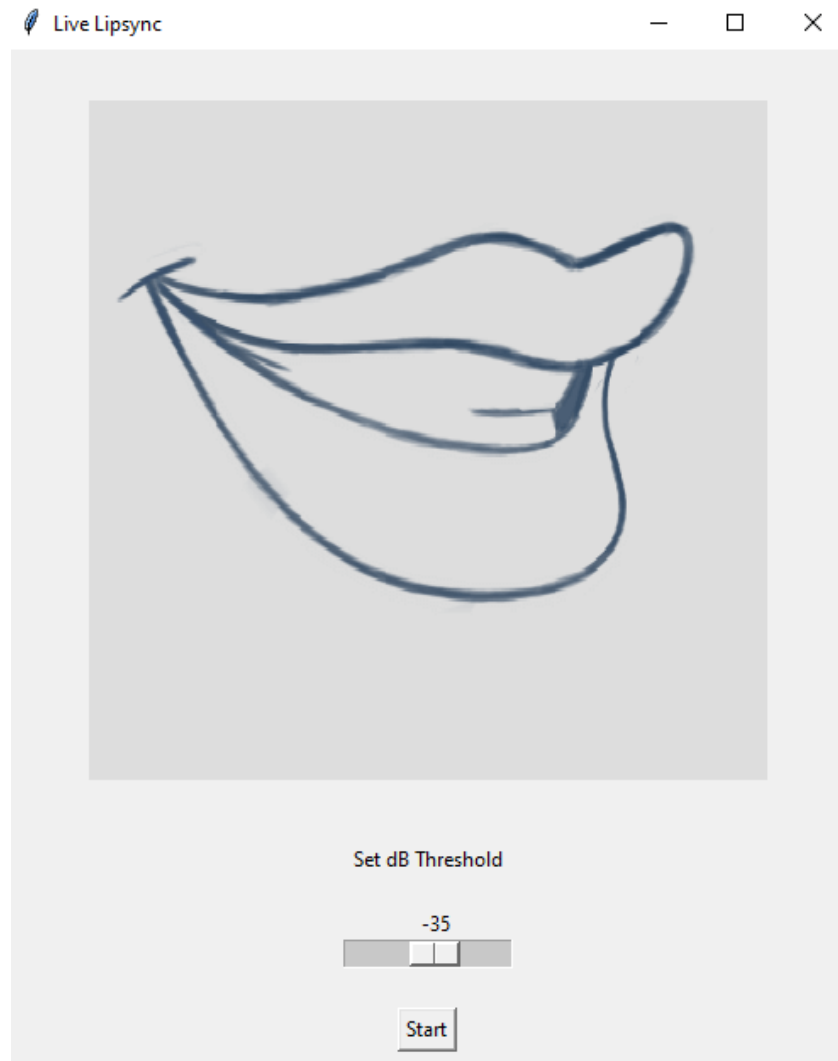
Comparing the post-filtering output of the model to the *ground truth* does not provide us with a good metric to evaluate performance. This is because Rhubarb lip sync does not operate with the same rules as the filters in this application, therefore the *real* ground truth is not Rhubarb’s output. The result this project tries to achieve is more like that of the human animators’ work, which does not match up with Rhubarb. Since I do not have access to human generated data, I cannot compare the model’s output to ground truth.

8 Animation

Once the model is loaded and configured to make predictions, it makes those at 100Hz as it was trained on that frequency. The raw output is quite chattery, so it was clear to me why the original authors implemented 2 types of filtering and additional downsampling. However, since the implementation of this solution most likely uses some kind of additional camera input to determine when the speaker is not speaking, I implemented an additional “silence mode” to make up for the fact that I’m only using audio input. Additionally, I implemented a noise reduction method, to be able to adapt to a wider range of microphones, not only the microphone used in the TIMIT audio recordings.

8.1 Interface

Figure 20: Simple demonstration interface for the animation



Source: Self made

The interface [Figure 20] is purely for demonstration, so it is simple, showcasing the basic functionality of the application. The “Set dB Threshold” slider controls the decibel threshold parameter of the inference code, to be able to adapt the sensitivity to any given microphone. Below is the Start/Stop button which enables or disables the animation and inference.

8.2 Noise reduction

In this application, noise reduction helps increase accuracy because the audio recordings the model was trained on had very low noise levels. Common microphones such as the one I’m using for

testing pick up some background noise, so eliminating this is useful. First, I take a one second sample of the noise levels, during this time, it is advised not to speak. This sample is the noise sample, which at this stage is just a simple recording of the raw audio. A second function then applies the noise reduction to each audio chunk. The reduction is done by `scipy.signal`'s `wiener` module, which, as the name indicates, applies Wiener filtering to the audio.

Wiener filtering takes a sample of the signal stream, then takes this known signal and uses it as an input in filtering a new signal that is being interfered with, estimating what it should be (MIT, 2010).

This results in a slightly cleaner audio signal going forward, technically enabling feature extraction to extract features that are closer to the ones seen in the training dataset. The visual impact of this is very low, however it helps eliminate noisy predictions when the speaker is quiet, by reducing the overall amplitude of the sound, therefore aiding in accidentally disengaging the next filter, the silence mode.

8.3 Silence mode

Silence mode introduces a check that enables this mode when the audio input does not hit a predefined decibel level. As long as silence is active, the entire prediction process is halted to avoid incorrect predictions. If the decibel levels exceed the threshold for a given amount of time, the mode disengages and starts the prediction process as long as the volume is high enough.

8.4 Reducing prediction noise

This filter utilizes a small amount of lookahead to filter out noisy predictions. Since the model is outputting predictions at 100 Hz, wrong predictions can slip through due to lack of model accuracy. To counteract this noise, the filter takes 10ms windows and checks if a viseme transition holds across the next 30ms window. It engages at every viseme change and keeps true transitions while replacing the false ones with the previous viseme. It adds 30ms latency due to the lookahead (Aneja & Li, 2019).

8.5 Downsampling to 24 FPS

At 100Hz (or 100 FPS), the animation can be quite chattery with rapid transitions, due to the errors made by the model during inference. Most TV animations are displayed at 24 FPS (Salmon, Armstrong, Jolly, Flynn, & Hammond, 2008), meaning that by downsampling the predictions, we can achieve the same effect. In this solution, the downsampling is not “true” as the predictions are still yielded at 100Hz, however, one frame at 24Hz roughly corresponds to 4 viseme predictions at 100Hz, so essentially downsampling means yielding the same visemes in batches of 4s, making sure that if there is a viseme that deviates in a batch, it gets replaced by majority vote.

8.6 Single frame viseme removal

In addition to the previous viseme filter, the single frame viseme removal filter applies a similar method, except at 24Hz. By previously denoising the predictions, this filter can work more accurately and requires less checks in the code, lightening the hit on latency. The filter is applied because animators generally do not display a viseme for less than 2 frames (Aneja & Li, 2019). To achieve the desired animation effect, it is important to follow the same guidelines as the animators would.

9 Limitations

When comparing this solution to the original research, one clear limitation emerges immediately. The fact that I did not use human labeled data, but instead used software-generated labels, means that the performance can only be as good as the labeling software, improved by the applied downsampling and filtering. Of course, this also means that the dataset was generated far quicker, and labeling was free, in contrast to the expensive process of hiring human animators.

Additionally, the model is only capable of representing active speech as the silence mode feature is quite rudimentary. Singing and laughing, and additional non-verbal expressions are also not displayed by this solution, as the training data did not include anything but speech. Based on information about the Adobe Ch software (Adobe Inc., 2024), these extra functions can be done via video input.

A limitation in the animation is that visemes are static images, there are no transition animations, this was simply not in the scope of this project. However, if the transition animation assets are available, they are easy to implement between each viseme.

Therefore, it is better to look at this application as a building block in a larger system, solely responsible for handling normal speech, and being supplemented by additional elements to create a believable animation.

10 Further development

There are a number of ways this application can be improved:

- Improved training data
- Additional audio features
- Further parameter testing
- Improve code modularity
- Eliminate performance bottlenecks

Improved training data:

Based on the experiments, believe the model would be able to learn much more effectively (relative to ground truth, when it is the human perception) if the dataset was labeled by human animators. Naturally, we expect to see the 2D animation as we have experienced it before, and this experience is based on work done by animators in cartoons we have seen. Automated labeling used in this paper cannot achieve the quality that human animators are capable of.

On the other hand, if automated labeling is used, the dataset can easily be expanded to include more audio sources to account for microphone quality, and more speakers, to introduce new dialects. Even if the application stays in English, speakers from more English-speaking countries could be recorded to create a more diverse dataset, which in turn improved generalization.

Additional audio features:

Depending on what performance evaluations like the confusion matrix indicate, additional audio features could be extracted to fix specific problems that exist in different spectral regions of the audio signal. However, latency should always be considered, because more features mean more processing done live during extraction, and larger tensors to be processed by the model.

Augment audio input:

Due to the difference in recording quality and dialects. The audio input can be changed to reflect the characteristics of the audio recordings used in the training dataset. This requires some research into the differences and the methods of augmentation.

Improve code modularity:

Time constraints meant that the code is not fully modular, there are some areas that could be improved. Ideally, the dataset generation, training, and inference process would all be easier to run and further modifications could be built on the project by adding more options for turning features on and off.

Eliminate performance bottlenecks:

Due to the experimental nature of this project, bottlenecks while training were not taken into consideration. The training process runs mostly on the GPU, which is preferred over the CPU due to the massive performance difference and the ability of parallel calculations. The performance increase can mean training times reduced by a factor of 3 or even more (Buber & Diri, 2018), (vinilv, 2023). However, the data generator architecture involves unavoidable CPU operations because it continually opens, reads from, and closes files in the operating system. The dataset in this case is only 4.3 Gigabytes, so it could be loaded into memory to massively increase training speeds. For easier diagnostics and debugging, I did not load it into the memory, this also means the dataset is quite flexible and can easily be changed just by adding new files. If in the future, more thorough testing is to be done, it is beneficial to load the data into the memory, to speed up each test iteration.

11 System specifications and tools

Computer the model was trained on:

- CPU: 13th Gen Intel® Core™ i7-13700KF
- GPU: NVIDIA GeForce RTX 4060 Ti
- RAM 32GB
- Microsoft Windows 10 22H2 build number: 19045.5131
- Trust GXT 242 Microphone

Tools used:

- Rhubarb Lip Sync (Wolf, 2022)
- Python 3.10.6
 - tkinter
 - PIL
 - librosa
 - torch
 - matplotlib
 - numpy
 - pyaudio
 - scipy
 - pydub
 - pandas
 - seaborn
 - sklearn

12 Conclusion

In this paper, I followed the methodology of the research it was based on (Aneja & Li, 2019), supplemented by my experience in building data pipelines and machine learning models and my research on audio processing, a field new to me. I faithfully recreated the system described by the researchers, with my own solutions to certain problems like data acquirement, and my interpretation of technical aspects of the application that were not explicitly described. Despite the results not achieving the desired quality, I believe the system would work well once the issues identified in the label generation are solved. To facilitate an easy adaptation to a new dataset, the codebase is written in a way that the new dataset can just simply replace the current one, and the whole process would work without any further adaptation, thanks to elements that ensure modularity, like dynamic label encoding that can adapt to different viseme sets, and the data generator's robust architecture ensuring compatibility with different datasets.

In addition, this research and the open-source codebase provides anybody, who wishes to create a system capable of real-time lip-sync for live 2D animation, with a much better starting point than the original paper due to the additional technical details and experiment descriptions which explored possible solutions and dead-end tracks to problems that may arise in the creation of an application such as this one.

The methodology used in this paper focused on using tools and solutions that were free and easy to access. Other than the TIMIT dataset, every part of this project is accessible to anybody. This also means attempting this same project with paid 2D lip-sync software would almost certainly mean better results, as long as the paid solutions offer higher quality lip-sync.

I invite any readers who may be interested in live 2D lip-sync to attempt to improve on the findings of this research and to write their own implementations in different programming languages or improve on the existing codebase. This kind of open-source application can be used as a building larger, more ambitious projects, increasing our capabilities to create innovative systems in the future. This project was solely possible thanks to the previous work of countless other experts who shared their knowledge, findings, and code to be available for everyone.

13 References

- Abdul, Z. K., & Al-Talabani, A. K. (2022, November). Mel Frequency Cepstral Coefficient and Its Applications: A Review. *IEEE Access*. doi:10.1109/ACCESS.2022.3223444
- Adobe Inc. (2024). *Animation lip sync*. Retrieved from Adobe: <https://www.adobe.com/creativecloud/video/discover/animation-lip-sync.html>
- Adobe Inc. (2024, November). *Wilmot Li*. Retrieved from Adobe Research: <https://research.adobe.com/person/wilmot-li/>
- Aneja, D., & Li, W. (2019, 8 19). Real-Time Lip Sync for Live 2D Animation. Retrieved from <https://arxiv.org/abs/1910.08685>
- Bäckström, T., Räsänen, O., Zewoudie, A., Zarazaga, P. P., Koivusalo, L., Das, S., . . . Vali, M. H. (2022). Introduction to Speech Processing. doi:10.5281/zenodo.6821775
- Brownlee, J. (2022, December). *Machine Learning Mastery*. Retrieved from <https://machinelearningmastery.com/cross-entropy-for-machine-learning/>
- Buber, E., & Diri, B. (2018). Performance Analysis and CPU vs GPU Comparison for Deep Learning. *2018 6th International Conference on Control Engineering & Information Technology (CEIT)*. doi:10.1109/CEIT.2018.8751930
- Collobert, R., Kavukcuoglu, K., & Farabet, C. (2011). Torch7: A matlab-like environment for machine learning. *Neural Information Processing Systems*.
- Dietterich, T. (1995, September). Overfitting and Undercomputing in Machine Learning. *ACM Computmg Surveys, Vol. 27*(No. 3). doi:10.1145/212094.212114
- Garofolo, J. S. (1993). *TIMIT Acoustic-Phonetic Continuous Speech Corpus*, LDC93S1. (Linguistic Data Consortium) Retrieved October 2024
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). In *The Elements of Statistical Learning* (Second Edition ed.). Springer.

- Huang, X., Allewa, F., Hon, H.-W., Hwang, M.-Y., Lee, K.-F., & Rosenfeld, R. (1993). The SPHINX-II speech recognition system: an overview. *Computer Speech & Language*, 137–148. doi:doi:10.1006/csla.1993.1007
- Huggins-Daines, D. (2024, October). *PocketSphinx*, 2.21.2. Retrieved October 2024, from Github: <https://github.com/cmusphinx/pocketsphinx>
- IBM. (2024, October 4). *IBM Topics*. Retrieved from Recurrent Neural Network (RNN): <https://www.ibm.com/topics/recurrent-neural-networks>
- IBM. (2024, July). *What is a loss function?* Retrieved from IBM Topics.
- Inkeaw, P. (2023). Mel Frequency Cepstral Coefficient. Retrieved November 19, 2024, from <https://www2.cs.science.cmu.ac.th/courses/204371/files/MFCC.pdf>
- McFee, B. (2024, May). *librosa/librosa: 0.10.2.post1*. (Zenodo, Producer) doi:10.5281/zenodo.11192913
- MIT. (2010). Wiener Filtering. *Introduction to Communication Control and Signal Processing, Chapter 11*.
- Norman, R. (2024, May). *What is VTubing, and why should brands take notice?* Retrieved from Amazon Ads: <https://advertising.amazon.com/blog/vtubing/>
- Prechelt, L. (2000). Early Stopping - But When? doi:10.1007/3-540-49430-8_3
- PyTorch Foundation. (2023). *PyTorch Documentation*, 2.5. Retrieved from <https://pytorch.org/docs/stable/index.html>
- PyTorch Foundation. (2024). *Saving and Loading Models*, 2.5.0+cu124. Retrieved from PyTorch Documentation: https://pytorch.org/tutorials/beginner/saving_loading_models.html
- Reallusion. (2024, November). *Motion Capture for 2D Animation*. Retrieved from <https://www.reallusion.com/cartoon-animator/motion-capture-animation/default.html>
- Sak, H., Senior, A., & Beaufays, F. (n.d.). Long Short-Term Memory Recurrent Neural Network Architectures for Large Scale Acoustic Modeling. Retrieved November 19, 2024, from <https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/43905.pdf>

Salmon, R., Armstrong, M., Jolly, S., Flynn, D., & Hammond, M. (2008). High Frame-Rate Television. *IBC2008*. BBC.

Schemenauer, N., Peters, T., & Hetland, M. L. (2001, June 23). *PEP 255 - Simple Generators*. Retrieved from Python Enhancement Proposals: <https://peps.python.org/pep-0255/>

Sepp, H., & Jürgen, S. (1997). Long Short-term Memory. *Neural computation*, 9. doi:10.1162/neco.1997.9.8.1735

vinilv. (2023, December). *Azure High Performance Computing (HPC) Blog*. Retrieved November 19, 2024, from Microsoft Community Hub: <https://techcommunity.microsoft.com/blog/azurehighperformancecomputingblog/exploring-cpu-vs-gpu-speed-in-ai-training-a-demonstration-with-tensorflow/4014242>

Werbos, P. (1990). Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78, 1550 - 1560. doi:10.1109/5.58337

Wikimedia Foundation. (2024, November 14). *Cross-entropy*. Retrieved from <https://en.wikipedia.org/wiki/Cross-entropy>

Wikimedia Foundation. (2024, November 19). *Root Mean Square*. Retrieved from https://en.wikipedia.org/wiki/Root_mean_square

Wolf, D. S. (2022, June 14). *Rhubarb Lip Sync*, 1.13.0. Retrieved October 2024, from Github: <https://github.com/DanielSWolf/rhubarb-lip-sync>

14 Table of Figures

Figure 1: High-level overview of the entire process	8
Figure 2: “The audio feature a consists of MFCC, log mean energy, and their first temporal derivatives”	12
Figure 3: Example of MFCC cepstrum	17
Figure 4: MFCC framework.....	20
Figure 5: High-level overview of LSTM architecture	33
Figure 6: LSTM Memory Block	34
Figure 7: Training loss & convergence over epochs	40
Figure 8: Learning rate change over epochs	42
Figure 9: Training & Validation accuracy over epochs	43
Figure 10: Training & Validation loss over epochs	44
Figure 11: First experiment confusion matrix	45
Figure 12: Third experiment confusion matrix	48
Figure 13: Fourth experiment confusion matrix	50
Figure 14: Distribution of visemes in the training dataset	51
Figure 15: Distribution of sentences by 'B' viseme content	52
Figure 16: Viseme distribution by frame for each animator	53
Figure 17: Feature spectrograms extracted from a recording in the TIMIT dataset	54
Figure 18: Log Energy spectrogram of my voice saying a sentence from the TIMIT dataset.....	55
Figure 19: MFCC spectrogram of my voice saying a sentence from the TIMIT dataset.....	55
Figure 20: Simple demonstration interface for the animation.....	58

15 Table of Tables

Table 1: Rhubarb viseme set	14
Table 2: File & Live extraction differences.....	24

Table of Appendices

1. Thesis topic draft
2. GitHub repository branch link
3. Demo videos

Appendices

4. GitHub Repository Branch:

https://github.com/Whaletail930/2DLipsync/tree/review_branch

5. Demo video: A short video that demonstrate the program while it is running. The audio is from the TIMIT dataset and is played through a speaker into a microphone.